Next Gen Graphics

for SnowRunner (SnowRunner Next Gen before October 2026). The guide: for beginners, and every card in depth

For Next Gen 1.0.2 and the changes since. The card texts are the program's own, from the build of 2026-10-24 16:05.
34 cards, 31 rows of options.

Contents

1. **Part 1. Next Gen for dummies**
    1. What Next Gen is
    2. What you need
    3. Put it in, step by step
    4. What to switch on
    5. Change your mind, or take it all out
    6. After a game update
    7. Other mods
    8. When something goes wrong
2. **Part 2. The window**
3. **Part 3. Every card at a glance**
    1. Light and shadow
    2. Reflections and water
    3. Atmosphere and image
    4. Scenery
    5. Interface
4. **Part 4. How it works, in depth**
    1. How Next Gen gets into the game
    2. Sun shadows: resolution, edges, softness, seams
    3. Contact shadows
    4. Headlight shadows
    5. Ambient occlusion: GTAO and the wide reach
    6. Bounce light
    7. Sky ambient and fill light
    8. Object reflections: the reflection pass (SSSR)
    9. Water: reflections, Fresnel, depth colour, crest glow
    10. Wet ground reflections and the glare cap
    11. Headlights in reflections
    12. Volumetric fog and sun shafts
    13. Tonemap, bloom and photo grade
    14. Reflection cubes from photographs
    15. Night sky, brighter stars, skies
    16. Smoke: glow, shape, softer edges
    17. Sharper smoke and particles
    18. Weather, and debris and splashes
    19. Scenery detail and grass reach
    20. Map days
    21. The load list: how a pak tells the game what it holds
    22. Next Gen logo and the start without the intro
    23. The engine: building, checking, undoing
    24. Known issues and fixes
5. **Part 5. What it changes on your PC**
6. **Part 6. Licence and credits**

Part 1. Next Gen for dummies

This part is for everyone: what Next Gen does, how to put it in and what to switch on. Part 3 has a page for every card in the same plain words; Part 4 explains how each one works, with the math.

What Next Gen is

Next Gen is a free graphics mod for SnowRunner. A small program, **SnowRunnerNextGen.exe**, changes a few of the game's own files on your PC: the shaders that draw light, shadow, water and smoke, and some of the game's pictures and settings. The game then draws sharper shadows, shade where things meet, reflections on paint and water, fog that glows toward the sun, smoke with a lit side and a truer night sky.

Apart from one small DLL, SnowRunner Shadows, nothing runs next to the game. You do not need ReShade or any other loader, and the program keeps a copy of every file it changes, so you can always go back.

What you need

- SnowRunner from Steam, Epic Games or the Microsoft Store, on Windows.
- Room on the system drive for the copies of the original files: up to about 4 GB, depending on what you switch on. shared.pak alone is 2 GB, and only the Scenery detail card changes it.
- The game closed while you install, change or remove anything. The program will not write while it runs.

Put it in, step by step

1. Download the newest zip and the sky pack, **SnowRunnerNextGen_Skies.zip**, from the project's releases page, **github.com/andizla/snowrunner-next-gen/releases**.
2. Close SnowRunner.
3. Unzip the folder anywhere, your desktop for example, unzip the sky pack into the same folder, and run **SnowRunnerNextGen.exe**. Nothing else needs to be installed.
4. The bar at the top shows the game folder the program found. If it found none, or the wrong one, click **Change** and pick the folder that holds SnowRunner.
5. Click **Defaults**, then **Apply**. The first Apply reads your game's own shaders and takes a minute or two. A window then says Installed.
6. Start the game.

To update, unzip the new version and its sky pack into a folder of their own and click **Apply**: an install made with an older version is updated in place. The window shows what is installed, so a card that is new in this version starts unticked; **Defaults** ticks every card as it comes.

What to switch on

Defaults gives you the recommended set: the 31 cards marked "on" in the table below. Start with it and play for a while.

The 3 cards that are off by default are off for a reason: they are experimental, a matter of taste, or they cost more frame rate. Each has its own page in Part 3 that says what it does, what you will notice and what it

costs. Switch one on, click Apply, and look in the game. With SnowRunner Shadows in, **F8** switches every shader effect off and on, so you can compare from the same spot.

Some cards have rows of buttons: a resolution, a strength, or which maps a change applies to. The values a card starts with are the recommended ones. Part 3 lists every row with its values.

Card	Default	What it does
Light and shadow		
SnowRunner Shadows	on	Hands the reflections and the bounce light the image they need, and can draw the sun's shadow texture at a higher resolution for finer shadow detail: 1x keeps the game's own size.
Shadow edges	on	Rebuilt straight shadow edges, with no texel stairs, at any resolution.
Blocker search	on	Shadows sharp at the contact and softer the farther they fall from what casts them.
Contact shadows	on	Fine sun shadows traced on screen where things touch or overlap, in detail the shadow texture is too coarse to hold.
Headlight shadows	on	Your headlights cast shadows: posts, trees and rocks block the beam instead of letting it light what stands behind them.
GTAO	on	Ground-truth ambient occlusion instead of the game's SSAO: soft contact shadows in corners, under trucks and in wheel arches.
Wide occlusion	on	GTAO's search reaches on to 6 metres, gently: soft shade under trucks, roofs and tree crowns, with the contact shadows unchanged.
Sky ambient	on	The fill light takes the sky's colour instead of a flat grey.
Fill light	on	Less of the flat light that fills in shade: shaded sides, the insides of trees and trucks in shadow get darker, while sunlit parts and the sky stay as they are.
Bounce light	off	Sunlit ground and paint light up what stands next to them, in their colour: under trucks, in wheel arches, at the feet of walls. Experimental: in places it comes out too strong.
Reflections and water		
Object reflections	on	Paint, glass and chrome mirror the scene around them instead of a painted sky.
Headlights in reflections	on	At night, paint that finds nothing on screen to reflect shows the warm light of the headlights.
Water reflections	on	Banks, trees and trucks mirrored in rivers, mud, lakes, seas and puddles, not just the sky; the water's colour gives way to the reflection at shallow angles.
Wet ground reflections	on	Wet mud and glossy ground mirror the scene around them: sharp in shiny patches, soft on rough mud.
River depth colour	on	Deeper water tints teal or brown from its own colour instead of fading grey.
Sun glow through waves	on	River waves between you and a low sun light up in the water's own colour, wherever the sun really reaches them.
Headlight glare cap	off	Tamer headlight shine on wet ground and puddles, with no long bright band across them.

Card	Default	What it does
Reflection cubes	on	The painted skies that paint, glass and chrome mirror, and that Sky ambient and Bounce light read the sky's and the ground's colour from, made anew from photographs: 25 cubes from 21 panoramas, for every region's clear days, weather days, dusk, dawn and night. Each keeps the brightness of the game's own cube, and the snow regions get snow.
Atmosphere and image		
Volumetric fog	off	No banding and a glow toward the sun. Sun shafts through trees and dust can be switched on: they are experimental and also show over your truck and other things close to the camera.
Weather	on	More of the game's own weather: moving cloud shadows on every map, showers that swell and ease, drizzle at dusk and night, horizon clouds, rain drawn farther out, fireflies at night and pollen by day.
Map days	on	Four regions have no days of their own and show another region's. Maine shows two snow regions': snow in the air and winter light on a map without snow. This gives each of their maps days that fit it, chosen map by map, with a row for each region.
Smoke glow	on	Exhaust and other smoke glows when the sun is behind it.
Smoke shape and softer edges	on	Smoke and dust puffs get a lit side and a shaded side, and sprites fade more softly where they meet the ground.
Sharper smoke and particles	on	The game's smoke, dust, spray and spark textures at twice their size, so particles stay crisp up close.
Debris and splashes	on	Leaves, grass, twigs and dirt thrown up by the wheels stay on the ground for half a minute and get pushed by the tyres, instead of vanishing after four seconds. Mud, water and snow splashes throw more drops that lie longer and leave more marks on the truck. Snow spray hangs in the air. Exhaust and dust hang longer too.
Night sky	on	NASA's star map in place of the game's star texture: the real sky's stars as fine points, with the glow of the Milky Way.
Skies	on	The painted skies of every region made anew from photographs of real skies: 49 of the game's sky domes, cloud bands and night layers, each at its own size and brightness, the photograph's sun taken out so the game's sun is the only one. The aurora is green, as a real one is. The clouds still drift.
Brighter stars	on	The stars of the night sky drawn brighter, so they stand out against the dark.
Tonemap	on	Stronger contrast and a lighter bloom.
Bloom soft cut	on	No haze from bright snow and sky.
Photo grade	on	Colour closer to a photo: greens, autumn leaves and the violet haze less loud, whites a little lower, no lens dirt over the sky. Built into the game's own colour grading.
Scenery		
Scenery detail	on	Scenery keeps its full detail farther out, so less pops in while you drive. All meshes makes shared.pak larger than 2 GB. Shown far away: plants (trees, bushes, rocks, twigs), or everything, no longer vanish at a set distance. Small shadows: what casts none, or loses it a few metres away, gets one.
Grass reach	on	Grass drawn farther out. Costs frame rate in grassy areas.
Interface		

Card	Default	What it does
Next Gen logo	on	NEXT GEN under the game's logo on the title screen, the first loading screen, the main menu, the pause menu and the map.

Change your mind, or take it all out

Tick or untick cards and click **Apply** again: the program builds the files for exactly what is ticked, every time from the game's original files. **Restore originals** puts every file back as it was before Next Gen changed it and takes SnowRunner Shadows out. The copies of the original files stay in **%LOCALAPPDATA%\SnowRunnerNextGen** (2 to 4 GB): to remove Next Gen for good, delete that folder by hand after Restore.

After a game update

A game update, or the store's check of the game files, puts the game's own files back, and Next Gen's changes are gone with them. Run SnowRunnerNextGen.exe and click **Apply** again.

One card needs this sooner than the others: with Map days set to "map by map" for Maine, Maine stops loading after such an update until you have run Apply again. The card's page says why.

Other mods

If another mod, a texture pack say, has changed one of the files Next Gen works on, the program asks before it writes. **Take it as the original**: Next Gen writes its changes over the file as it is now, and Restore later puts that modded state back. **Leave it out**: that file and its cards stay as they are, and the rest is applied.

When something goes wrong

Every Apply and Restore is written to **install.log** in **%LOCALAPPDATA%\SnowRunnerNextGen**: what the program found and each step it took. In the result window, **Show the log** opens that folder. Send the file along when you report a problem.

Support, questions and talk about the mod: the channel on EnthusiastRunner's Discord, with thanks to EnthusiastRunner for it.

Part 2. The window

Next Gen Graphics installs, changes and removes the graphics modules of this project, all from one window.

Pick

Each card is one module. Ticking a module that needs another ticks that one too, and unticking one unticks what needs it. The (?) on a card tells you what it does. Defaults picks the recommended set.

Apply

Builds the game's files for exactly what is ticked, from the game's original files, and installs them. To take a module out, untick it and apply again.

Restore originals

Puts the files back as they were before Next Gen Graphics changed them and takes SnowRunner Shadows out.

Backups

Before the first change, shader.pak, shared.pak, initial.pak, boot.pak and gfx.pak are kept as they are in %LOCALAPPDATA%\SnowRunnerNextGen, and every build starts from those copies. Next to the game's files the program leaves a small note, SnowRunnerNextGen.json, naming the ones that hold its changes. Should the kept copies ever be lost (a new Windows install, say), the note lets it see that those files are not the game's own: it then changes nothing and asks you to have the store check the game's files first.

Other mods

When another mod (a texture pack, say) or a game update has changed one of those files since, Apply and Restore ask first. Take it as it is now as the original, and Next Gen Graphics writes its changes over it; Restore then puts it back in that modded state, the other mod's changes included, not the game's own file. Or leave it out: the file and its modules stay as they are.

Before you start

Close the game: its files cannot be changed while it runs.

When something goes wrong

Every Apply and Restore is written to install.log in %LOCALAPPDATA%\SnowRunnerNextGen: what the program found and each step it took. After a stop or a note, Show the log in the result window opens that folder. Send the file along when you report a problem.

Comparing in the game

With SnowRunner Shadows installed: F8 switches every shader effect off and on, F4 the contact shadows, F5 the ambient occlusion pass between half and full size, F6 the reflection pass, F9 the reflections on wet ground, F10 the bounce light, and F11 GTAO against the game's SSAO. The first press of F8 or F11 takes a moment: the game's own shaders are made then. The keys do nothing while Alt, Ctrl or Shift is held. With Next Gen Gameplay or Next Gen Tire Inflation installed, the same switches are on a Graphics tab next to theirs in their NEXT GEN menu (F2), and these keys stay silent, as other mods use some of them; KeysWithMenu=1 in SnowRunnerShadows.ini keeps them. Next Gen Graphics alone has no menu, and its keys work as before.

Part 3. Every card at a glance

Light and shadow

SnowRunner Shadows

Shadows · on by default · in depth: How Next Gen gets into the game, Sun shadows: resolution, edges, softness, seams

Hands the reflections and the bounce light the image they need, and can draw the sun's shadow texture at a higher resolution for finer shadow detail: 1x keeps the game's own size.

Row	Values (the default in bold)
Resolution	**1x** · 2x · 3x · 3.5x
Self-shadow fix	**on** · off

WHAT IT DOES

The sun's shadows are drawn into one shadow texture before every frame. This add-on hands the scene's picture to the bounce light and the reflections, which is why they need it, and can make the shadow texture larger, up to 3.5 times in each direction, so every shadow is drawn with more pixels. It is one small file next to the game (hid.dll) and changes none of the game's own files.

WHAT YOU WILL NOTICE

At 2x and above: finer shadow detail on trees, fences, cranes and trucks, also some way off.

WHAT IT COSTS

At 1x the shadow texture costs nothing extra. A larger one costs graphics time while the shadows are drawn, and video memory (a few hundred MB at 3.5x). Measured at 4K on an RTX 4080 with every other module on: 2x 43 fps, 3x 37 fps, 3.5x 33 fps, against 49 fps at the game's own size.

OPTIONS

Resolution: 1x keeps the game's own shadow texture, and Shadow edges keeps its edges straight; 2x, 3x and 3.5x draw it larger for finer detail, at the frame rates above. Self-shadow fix: keeps the game's shadow bias in step with the larger texture, so steep rock faces do not shadow themselves in patches. Leave it on.

GOOD TO KNOW

In the game, F8 switches every shader effect of this installer off and on again, for a quick comparison. The larger shadow texture stays.

Shadow edges

Shadows · on by default · in depth: Sun shadows: resolution, edges, softness, seams

Rebuilt straight shadow edges, with no texel stairs, at any resolution.

Row	Values (the default in bold)
Method	**rebuilt edges** · 16-tap filter
Cascade seams	**blended** · off

WHAT IT DOES

The sun's shadow texture has a limited number of texels, so the game draws the edge of a shadow as a staircase of them. Rebuilt edges follows each edge across the texels and draws the straight line they make, so edges come out straight and clean at any resolution, even at the game's own. The 16-tap filter, the earlier method, smooths the edges with 16 samples on an even grid instead.

WHAT YOU WILL NOTICE

Straight, clean edges on the shadows of buildings, trucks, fences and cranes, without the stair pattern.

WHAT IT COSTS

Less than the 16-tap filter: about 0.7 ms per frame faster like for like, measured at 4K on an RTX 4080. Rebuilt edges at 1x ran at 64.8 fps where the 16-tap filter at 2x ran at 59.7.

OPTIONS

Method: rebuilt edges (the default) or the 16-tap filter, one or the other. Cascade seams: the game draws sun shadows in steps of detail (cascades) that change at set distances, which can show as a line where the shadows turn coarser. Blended dithers the last few percent of each step into the next, so the line no longer shows. It comes with rebuilt edges only.

GOOD TO KNOW

Round shadows can come out slightly angular, and very soft shadow ends keep a little of the stair pattern.

CREDITS

The rebuilt edges follow revectorization-based shadow mapping by Macedo and Apolinário (2016) and shadow map silhouette revectorization by Bondarev (2014).

Blocker search

Shadows · on by default · in depth: Sun shadows: resolution, edges, softness, seams

Shadows sharp at the contact and softer the farther they fall from what casts them.

WHAT IT DOES

Real sun shadows are sharp where an object meets the ground and get softer the farther they fall from it. This looks up how far each shadowed point is from what blocks the sun, and softens the edge to match.

WHAT YOU WILL NOTICE

Crisp contact shadows under tyres and posts, long tree shadows with soft ends.

WHAT IT COSTS

Small: one search of 16 samples where shadows fall.

GOOD TO KNOW

Works together with Shadow edges, with either method.

Contact shadows

Shadows · on by default · needs SnowRunner Shadows · in depth: Contact shadows

Fine sun shadows traced on screen where things touch or overlap, in detail the shadow texture is too coarse to hold.

WHAT IT DOES

The sun's shadow texture is too coarse for small things: a door handle, a mirror or a stone casts no shadow of its own, and a wheel seems to float just above the ground. For every pixel this follows a short ray towards the sun through what is on screen and adds the shadow of whatever blocks it.

WHAT YOU WILL NOTICE

Wheels and trucks sit on the ground, and small parts such as handles, mirrors and bolts cast their own shadows.

WHAT IT COSTS

Some graphics time: one more pass over the screen.

GOOD TO KNOW

Needs SnowRunner Shadows, which runs the pass. Only what is on screen can cast these shadows. In the game, F4 switches it off and on.

CREDITS

A modified copy of Bend Studio's screen-space shadows (Copyright 2023 Sony Interactive Entertainment, Apache License 2.0).

Headlight shadows

Shadows · on by default · needs SnowRunner Shadows · in depth: Headlight shadows

Your headlights cast shadows: posts, trees and rocks block the beam instead of letting it light what stands behind them.

WHAT IT DOES

The game lights everything in your headlights' cone, even what stands behind a post or a tree. For every lit pixel this walks from it towards each of the two lamps through what is on screen, and dims the headlight's light where something stands in the way.

WHAT YOU WILL NOTICE

At night, posts, trees, rocks and other trucks throw long shadows away from your lamps, their edges half lit where only one lamp reaches.

WHAT IT COSTS

Some graphics time where the headlights fall: up to 48 depth reads for each lit pixel.

GOOD TO KNOW

Needs SnowRunner Shadows, which supplies the depth. Only what is on screen can cast these shadows, and nothing closer than 2.5 m to the lamps. Tested in the game and still being tuned.

GTAO

Light · on by default · in depth: Ambient occlusion: GTAO and the wide reach

Ground-truth ambient occlusion instead of the game's SSAO: soft contact shadows in corners, under trucks and in wheel arches.

Row	Values (the default in bold)
Quality	**half size** · full size

WHAT IT DOES

Ambient occlusion is the soft shadow where things meet: under a truck, between tyres, in corners. The game's own version is an old depth-only SSAO with a radius that covers the same share of the screen at any distance, which draws a dark outline along every edge and a grey film on flat ground. GTAO (ground truth ambient occlusion) looks along several directions around every pixel, finds how much of the sky the geometry nearby leaves open, and darkens by exactly that, within 1.5 metres in the world.

WHAT YOU WILL NOTICE

No dark outlines along grilles, treads and railings, clean open ground, and soft shadow where things really close in: between twin tyres, inside the frame, where a wheel meets the ground. Thin things such as poles, cables and railings no longer carry a halo.

WHAT IT COSTS

More graphics time than the game's own pass at full size. At half size, the default, SnowRunner Shadows draws the pass at half the resolution, then blurs and scales it up: at 4K on an RTX 4080 that took the frame rate from 63.7 to 75.0 fps.

OPTIONS

Quality: half size (the default, much faster) or full size (the sharpest). Half size needs SnowRunner Shadows, which draws the pass; without it the pass runs at full size.

GOOD TO KNOW

Ambient occlusion has to be on in the game's video settings. Bounce light is built on it. In the game, F11 switches between GTAO and the game's SSAO, and F5 between half and full size.

CREDITS

Ground truth ambient occlusion by Jorge Jimenez, Xian-Chun Wu, Angelo Pesce and Adrian Jarabo (2016). The horizon integration follows Intel's XeGTAO by Filip Strugar and Steve Mccalla (MIT).

Wide occlusion

Light · on by default · needs GTAO · in depth: Ambient occlusion: GTAO and the wide reach

GTAO's search reaches on to 6 metres, gently: soft shade under trucks, roofs and tree crowns, with the contact shadows unchanged.

WHAT IT DOES

GTAO looks, in several directions around every point, for what hides the sky within 1.5 metres. This lets the same search go on to 6 metres, and what it finds out there darkens only gently: half as much at 1.5 metres, nothing at 6. So the ground under a truck, the walls under a roof and the inside of a tree crown get a soft shade, while the contact shadows stay exactly as GTAO draws them.

WHAT YOU WILL NOTICE

A soft pool of shade under trucks and trailers, shade under roofs and overhangs, a little more depth inside trees.

WHAT IT COSTS

Little: about 1 ms per frame at 4K (measured on an RTX 4080). Each pixel looks far out in one direction and shares what it finds with its neighbours.

GOOD TO KNOW

Needs GTAO. In the game, F11 switches between GTAO (with this) and the game's own SSAO.

Sky ambient

Light · on by default · in depth: Sky ambient and fill light

The fill light takes the sky's colour instead of a flat grey.

WHAT IT DOES

Shaded places are lit by one fill colour per map and time of day. This makes the fill light take the colour of the sky above: bluer under a clear sky, warmer toward a low sun. How bright it is stays the game's choice.

WHAT YOU WILL NOTICE

Shade that matches the sky instead of a flat grey.

WHAT IT COSTS

Tiny: a few reads of the sky picture.

Fill light

Light · on by default · in depth: Sky ambient and fill light

Less of the flat light that fills in shade: shaded sides, the insides of trees and trucks in shadow get darker, while sunlit parts and the sky stay as they are.

Row	Values (the default in bold)
Strength	70 % · **55 %** · 85 %

WHAT IT DOES

Shade in the game is lit by a strong, even fill light: on many maps it is nearly as bright as the sun, so shaded sides, the insides of trees and trucks in shadow look flat and washed out. This turns that fill light down in the day, dusk and dawn light of every map, so shade gets the depth it has in a photo. Night, the garage and the menus keep theirs. Because less fill light darkens the whole picture, the game's eye adaptation would lift it all back up and sunlit parts would end up brighter than before; so the exposure of the same light is lowered with it, which keeps sunlit parts about where they were.

WHAT YOU WILL NOTICE

Deeper shade under trees and on the shaded sides of trucks and rocks, more depth inside foliage, and sunlit parts that stand out more.

WHAT IT COSTS

None: it changes how much of the game's own fill light is used.

OPTIONS

Strength: how much of the fill light stays. 55 %, the default, gives deep shade, 70 % a little less, 85 % a light touch.

GOOD TO KNOW

It changes initial.pak, together with Grass reach, which it keeps. Other mods that change initial.pak change the same file. Sky ambient still colours what fill light is left.

Bounce light

Light · off by default · needs GTAO, SnowRunner Shadows · in depth: Bounce light

Sunlit ground and paint light up what stands next to them, in their colour: under trucks, in wheel arches, at the feet of walls. Experimental: in places it comes out too strong.

WHAT IT DOES

Light bounces. Sunlit snow brightens the underside of a truck, and red paint tints the ground beside it. This adds that bounce at short range, up to about 1.5 metres, gathered from the scene's picture every frame.

WHAT YOU WILL NOTICE

Less black under trucks and in wheel arches on bright days, and a faint spill of colour from paint and mud.

WHAT IT COSTS

Under 1 ms per frame at 4K (measured on an RTX 4080): it gathers light in the ambient occlusion pass and adds it to every lit material.

GOOD TO KNOW

Off by default and experimental: in places the bounce comes out too strong. On snow and ice it takes its colour from the reflection cube, so keep Reflection cubes on with it. Needs GTAO, whose pass it works in, and SnowRunner Shadows, which hands it the scene's picture. In the game, F10 switches it off and on.

Reflections and water

Object reflections

Reflections · on by default · needs SnowRunner Shadows · in depth: Object reflections: the reflection pass (SSSR)

Paint, glass and chrome mirror the scene around them instead of a painted sky.

Row	Values (the default in bold)
Method	**reflection pass** · march only

WHAT IT DOES

The game's paint, glass and chrome reflect a ready-made picture of the sky, so trucks never mirror what is around them. This makes them reflect what is on screen: the road, the trees and other trucks.

WHAT YOU WILL NOTICE

Paint and windows mirroring their surroundings, most at shallow angles, and chrome showing the ground.

WHAT IT COSTS

About 0.3 to 0.4 milliseconds per frame at 4K with the reflection pass.

OPTIONS

Method: the reflection pass traces the reflections of the whole screen in one go (in SnowRunner Shadows), blurs them by how rough each surface is and steadies them over frames; windows, and whatever the pass misses, fall back to the march. March only lets every material search the last frame on its own: the older method, kept as a choice.

GOOD TO KNOW

Only what is on screen can be reflected, so what is behind the camera shows the sky picture as before. In the game, F6 switches the reflection pass off and on.

CREDITS

The reflection pass follows AMD FidelityFX SSSR (MIT), with Morgan McGuire and Michael Mara's screen-space ray tracing, Eric Heitz's visible-normal sampling and Marco Salvi's variance clipping.

Headlights in reflections

Reflections · on by default · needs Object reflections · in depth: Headlights in reflections

At night, paint that finds nothing on screen to reflect shows the warm light of the headlights.

WHAT IT DOES

With Object reflections, a hood or roof whose reflection finds nothing on screen shows a picture of the sky. At night this adds the headlights' warm light to those reflections, where the mirrored ray crosses the headlight beam.

WHAT YOU WILL NOTICE

At night, hoods and roofs pick up the glow of the truck's own lamps.

WHAT IT COSTS

None while the lamps are off. With two lamps on, at most 0.15 ms per frame at 4K if glossy paint filled the whole screen (measured on an RTX 4080), so far less in a real frame.

GOOD TO KNOW

Needs Object reflections with the reflection pass.

Water reflections

Water · on by default · needs SnowRunner Shadows · in depth: Water: reflections, Fresnel, depth colour, crest glow

Banks, trees and trucks mirrored in rivers, mud, lakes, seas and puddles, not just the sky; the water's colour gives way to the reflection at shallow angles.

Row	Values (the default in bold)
Sun glitter in shade	**the game's** · off in shadow

WHAT IT DOES

Rivers, mud, lakes, seas and the puddles on the roads reflect what is on screen: banks, trees and trucks, not just the sky. The water's own colour gives way to the reflection at shallow angles, as with real water.

WHAT YOU WILL NOTICE

Mirrored banks and trees in calm water, trucks in the road puddles.

WHAT IT COSTS

Small, and only where water is on screen.

OPTIONS

Sun glitter in shade: the game leaves a share of the sun's glitter on water that lies in the sun's shadow, so a glitter path can run on under a cliff or a bridge. Off in shadow puts it out where the water's own shadow test says the sun is hidden, in the 64 lake and 8 river shaders drawn with the sun's shadow map. Off by default and experimental: where it was tried, the game's test said the water was lit and nothing changed.

GOOD TO KNOW

Needs SnowRunner Shadows for the scene's picture. What is off screen falls back to the game's own reflection.

Wet ground reflections

Reflections · on by default · needs SnowRunner Shadows · in depth: Wet ground reflections and the glare cap

Wet mud and glossy ground mirror the scene around them: sharp in shiny patches, soft on rough mud.

WHAT IT DOES

Wet ground in the game reflects only a ready-made picture of the sky. This lets it reflect what is really around it, trucks, trees and banks, from the scene SnowRunner Shadows hands over: sharp where the ground is glossy, blurred where it is rough, the way wet mud mirrors. Rough dry ground is left alone.

WHAT YOU WILL NOTICE

Trucks and trees mirrored softly in wet mud and in shiny wet patches, most at shallow angles.

WHAT IT COSTS

Some frame rate where a lot of wet ground is on screen: each glossy pixel searches the scene.

GOOD TO KNOW

Needs SnowRunner Shadows. The road puddles are water, which Water reflections covers. What is off screen falls back to the sky picture. In the game, F9 switches it off and on.

River depth colour

Water · on by default · in depth: Water: reflections, Fresnel, depth colour, crest glow

Deeper water tints teal or brown from its own colour instead of fading grey.

WHAT IT DOES

Deep river water fades to a flat grey in the game. This tints it by its own colour as it gets deeper: teal in clear rivers, brown in muddy ones.

WHAT IT COSTS

Tiny.

Sun glow through waves

Water · on by default · needs Water reflections · in depth: Water: reflections, Fresnel, depth colour, crest glow

River waves between you and a low sun light up in the water's own colour, wherever the sun really reaches them.

WHAT IT DOES

Looking across water towards a low sun, light comes through the thin tops of the waves and glows in the water's colour: teal in clear water, brown in muddy. This adds that light to the rivers, strongest looking straight into the light, and only where the sun really reaches the water: it reads the game's own sun shadows, so water under trees stays dark.

WHAT YOU WILL NOTICE

Glowing wave tops on rivers against a low sun.

WHAT IT COSTS

Small: one more look at the sun's shadow on river water.

GOOD TO KNOW

The rivers whose shader knows where the sun's shadow falls (12 of the game's 20 river shaders); lakes and seas not yet. Needs Water reflections.

Headlight glare cap

Water · off by default · in depth: Wet ground reflections and the glare cap

Tamer headlight shine on wet ground and puddles, with no long bright band across them.

WHAT IT DOES

At night, headlights draw a long, very bright band across wet ground and puddles. This caps that highlight, so wet ground still shines, without the band.

WHAT IT COSTS

None.

GOOD TO KNOW

Off by default for now: it was tuned before the reflections changed. Tick it if the band bothers you.

Reflection cubes

Water · on by default · in depth: Reflection cubes from photographs

The painted skies that paint, glass and chrome mirror, and that Sky ambient and Bounce light read the sky's and the ground's colour from, made anew from photographs: 25 cubes from 21 panoramas, for every region's clear days, weather days, dusk, dawn and night. Each keeps the brightness of the game's own cube, and the snow regions get snow.

WHAT IT DOES

Each state names a reflection cube, a small painted sky with its ground, which paint, glass and chrome mirror where the scene has nothing nearer, and which Sky ambient and Bounce light read the sky's and the ground's colour from. The game's states name 26 of them, in five families whose files are named after Michigan, Taymyr, Mexico, Alaska and Scandinavia. This replaces 25 with cubes made from photographs: clear days, weather days, dusk, dawn and night. The sky, the sky straight overhead and the ground land at the brightness of the game's own cube, and at dusk, at dawn and at night at its colours too, so every hour keeps its light. In the two snow families the ground is snow at its own brightness by day, at dusk and at dawn. Their night cubes are winter day pictures graded to night. The garage keeps the game's cube, which shows the garage itself. While the cubes are in, Kola's and Yukon's clear days all name the snow cube, and the few states of Wisconsin and North Carolina that named a snow cube name one of their own region's.

WHAT YOU WILL NOTICE

Chrome and glass mirroring a photographed sky and ground instead of a hazy painting, on every map and at every hour, and the shade and the bounce light taking their colour from it.

WHAT IT COSTS

Nothing in the game: the cubes have the size of the game's own. 12 MB more in the download.

GOOD TO KNOW

It changes boot.pak, together with Photo grade, Sharper smoke and particles and Night sky, which it keeps. The cubes are made with engine\tools\make_cube.js from 21 CC0 panoramas by Poly Haven. THIRD_PARTY_NOTICES.md names them and their authors.

Atmosphere and image

Volumetric fog

Atmosphere · off by default · in depth: Volumetric fog and sun shafts

No banding and a glow toward the sun. Sun shafts through trees and dust can be switched on: they are experimental and also show over your truck and other things close to the camera.

Row	Values (the default in bold)
Sun shafts	**off** · on

WHAT IT DOES

The game draws its fog in steps that show as bands. This breaks the bands up and adds a glow toward the sun. Sun shafts: the fog also sees the sun's shadows, so shafts of light show through trees and dust.

WHAT YOU WILL NOTICE

Smooth fog and a bright haze toward the sun. With the sun shafts on, fog and a low sun behind trees: rays of light.

WHAT IT COSTS

Small.

GOOD TO KNOW

Off by default. The sun shafts on the card are off too unless you switch them on. They are experimental: the fog is drawn without knowing what stands in front of it, so with a low sun the rays also show over your truck and other things close to the camera. The rays need fog and a low sun. On a clear day there is little to see.

Weather

Atmosphere · on by default · in depth: Weather, and debris and splashes

More of the game's own weather: moving cloud shadows on every map, showers that swell and ease, drizzle at dusk and night, horizon clouds, rain drawn farther out, fireflies at night and pollen by day.

Row	Values (the default in bold)
Cloud shadows	**on** · off
Showers	**swell and ease** · as they are
Evening drizzle	**on** · off
Horizon clouds	**on** · off
Far rain and snow	**on** · off
Fireflies	**on** · off
Pollen	**on** · off
Drifting clouds	**on** · off
Far land	**on** · off

WHAT IT DOES

The game cycles each region through day states that can carry rain or snow, and on two maps moving cloud shadows. This adds more of that, from the game's own weather system. Cloud shadows: every map gets them, drawn from the game's own shadow textures. Showers: rain and snow swell and ease over time instead of holding one strength. Evening drizzle: the rain regions get light rain at dusk, at night and at dawn, as the snow regions already have flurries then. Horizon clouds: the cloud layer on the horizon that 17 sky files hold switched off, switched on. Far rain and snow: drawn out to 1.5 times the distance, 48 metres instead of 32. Fireflies: fireflies at night in the rain regions, drawn with the game's own firefly sprite. Pollen: pollen drifting through the clear days of the same regions. These two are weather types from the game's later engine, written in as two new files. Drifting clouds: a sky is made of layers that turn, each at a speed the day state sets, and in most states the nearest band of clouds and some of the sky pictures stand still while the bands behind them move. Where a cloud picture stands still it now turns slowly, about 150 settings in 17 sky files. The stars, the sun and moon, the aurora, the Milky Way and a painted sunset stay where they are. Far land: 46 states, most of them on Kola, Yukon, Quebec and Scandinavia, stop drawing the ground at 800 metres although their distant fog reaches farther, so in clear air the land ends short. There the ground is drawn as far as the fog reaches, 3000 metres at most.

WHAT YOU WILL NOTICE

Cloud shadows drifting over every map, and rain and snow that come and go.

WHAT IT COSTS

Little. The cloud shadows are drawn the way the game draws them on the maps that have them; the other parts change numbers the game reads anyway.

OPTIONS

Each part on its own, all on by default. Far land draws more ground; its cost has not been measured yet. With Fireflies and Evening drizzle both on, the nights of the rain regions take the fireflies and the drizzle keeps to dusk and dawn.

GOOD TO KNOW

> It changes initial.pak, together with Grass reach, Fill light and Brighter stars, which it keeps. Other mods that change initial.pak change the same file: Apply asks before it builds on it.

Map days

Atmosphere · on by default · in depth: Map days, The load list: how a pak tells the game what it holds

Four regions have no days of their own and show another region's. Maine shows two snow regions': snow in the air and winter light on a map without snow. This gives each of their maps days that fit it, chosen map by map, with a row for each region.

Row	Values (the default in bold)
Maine	Michigan's days · the game's · **map by map**
British Columbia	North Carolina's days · the game's · **map by map**
Austria	both maps alike · the game's · **map by map**
Washington	Michigan's autumn · the game's · **map by map**

WHAT IT DOES

Each map's level file names the day states it cycles through and its sky file. Four regions have none of their own and name another region's. Maine: its first map names Yukon's and its second Alaska's, two snow regions, so Maine gets their snowfall at dusk and at night and their light. This gives both maps Michigan's days, weather and sky. British Columbia: its maps name Michigan's, and the row gives them North Carolina's, which the game itself gives Washington, the region next to it. Austria: its first map names another region's days than its second, and the row gives the first the second's. Washington, an autumn region: its three maps name North Carolina's summer, and the row gives them Michigan's autumn.

WHAT YOU WILL NOTICE

On Maine: no snow in the air, and the light, weather and sky of an autumn forest map.

WHAT IT COSTS

Nothing.

OPTIONS

A row for each region, each map by map by default: days chosen for each map on its own. The other values give a region's maps one other region's days, or leave the game's. Maine: The Lowland has no snow on the ground and gets Michigan's wet days; Yellowrock National Forest has snow and ice and gets Yukon's spring with flurries instead of blizzards. Both are sets of Maine's own, so each map can be tuned on its own later. British Columbia: Duncan Bay, flooded, gets Wisconsin's summer with its heavy rain; North Peak National Park gets North Carolina's. Austria: Unterfeld, the farmland, gets Tennessee's summer; Foothills keeps the game's North Carolina. Washington is an autumn region whose three maps show North Carolina's summer: its row gives them Michigan's autumn, and map by map Selten River Mountains gets Zurdania's mountain autumn.

GOOD TO KNOW

It changes a few names in your own level files (level_us_06_01.pak and the like), in place, and Restore writes the game's names back, after which the files are as they were, byte for byte. A level file that another mod changed is left as it is. A game update or the store's file check brings the game's names back: Apply again. Most values only name days the game ships. Maine map by map also adds twenty small files to initial.pak and names them in its load list. When a game update or the store's file check replaces initial.pak while Maine's level files still name them, Maine stops loading until Apply has run again.

Smoke glow

Atmosphere · on by default · in depth: Smoke: glow, shape, softer edges

Exhaust and other smoke glows when the sun is behind it.

Row	Values (the default in bold)
More kinds of smoke	**off** · on

WHAT IT DOES

Exhaust, smoke, dust and steam glow when the sun is behind them, the way real smoke lights up against the sun.

WHAT YOU WILL NOTICE

Exhaust and dust clouds lighting up when you look toward a low sun.

WHAT IT COSTS

Tiny.

OPTIONS

The glow reaches 162 of the game's 205 particle shaders. More kinds of smoke adds eight that light their puffs the same way; it is off by default because it has not been looked at in the game yet.

Smoke shape and softer edges

Atmosphere · on by default · in depth: Smoke: glow, shape, softer edges

Smoke and dust puffs get a lit side and a shaded side, and sprites fade more softly where they meet the ground.

WHAT IT DOES

The game lights each smoke or dust puff as one flat colour. This gives the 80 lit puff sprites shape pixel by pixel: a lit side towards the sun, a shaded far side, lighter tops and slightly darker dense cores. The 110 sprites that fade where they cut into the ground or a truck now fade over twice the distance, so the hard line where a puff meets a surface is gone.

WHAT YOU WILL NOTICE

Rounder, more solid exhaust and dust clouds, and no sharp edge where smoke meets the ground.

WHAT IT COSTS

Small: a few more instructions in the particle shaders.

GOOD TO KNOW

Works with or without Smoke glow.

Sharper smoke and particles

Atmosphere · on by default · in depth: Sharper smoke and particles

The game's smoke, dust, spray and spark textures at twice their size, so particles stay crisp up close.

WHAT IT DOES

Smoke, dust, spray, sparks, leaves and the other particles are drawn from small textures that the game stretches, so up close they look soft and blocky. This puts in the same textures at twice their size, upscaled from the game's own with NVIDIA's DLSS 5 Visual Enhancer, inside boot.pak next to the photo grade.

WHAT YOU WILL NOTICE

Crisper exhaust, dust clouds and spray near the camera, with the same colours and shapes.

WHAT IT COSTS

Video memory for the larger textures. The installer is 114 MB larger for the set.

GOOD TO KNOW

It changes boot.pak, together with Photo grade and Night sky, which it keeps. Other mods that change boot.pak, such as a texture pack, are kept too: Apply asks before it builds on them.

Debris and splashes

Atmosphere · on by default · in depth: Weather, and debris and splashes

Leaves, grass, twigs and dirt thrown up by the wheels stay on the ground for half a minute and get pushed by the tyres, instead of vanishing after four seconds. Mud, water and snow splashes throw more drops that lie longer and leave more marks on the truck. Snow spray hangs in the air. Exhaust and dust hang longer too.

Row	Values (the default in bold)
Splash drops	**4x** · 8x

WHAT IT DOES

What the wheels throw up is gone in a few seconds: leaves, grass and twigs after four, the drops of a mud or water splash after two. This keeps it around. Leaves, grass, twigs, dirt and stones live 30 to 40 seconds and come to rest on the ground, where the tyres push them. Mud, water and snow splashes throw four times the drops, the drops lie twice as long, the splash lasts twice as long and leaves twice the mud and water marks on the truck. Exhaust, dust and steam hang in the air half as long again. Snow spray hangs in the air as it does in life: the snow drops fall at half the gravity with less air resistance and live 15 seconds, and the snow splashes last three times as long. The spray a wheel throws spinning in water flies about 2.5 times as far and is half as big again: it and its drops have less air to slow them, and they live longer. Everywhere else, air resistance, gravity, wind and the sizes stay the game's.

WHAT YOU WILL NOTICE

A trail of leaves and torn grass behind the truck that stays, mud splashes with more and longer-lived drops, big water spray off a spinning wheel, exhaust that lingers.

WHAT IT COSTS

Frame rate where much is thrown up: about ten times as many pieces are alive at once, and a splash has four times the drops. At 8x drops, twice that.

OPTIONS

Splash drops: 4x or 8x the game's drops per splash.

GOOD TO KNOW

It changes initial.pak, together with Grass reach, Fill light, Brighter stars and Weather, which it keeps. Other mods that change initial.pak change the same file: Apply asks before it builds on it.

Night sky

Atmosphere · on by default · in depth: Night sky, brighter stars, skies

NASA's star map in place of the game's star texture: the real sky's stars as fine points, with the glow of the Milky Way.

WHAT IT DOES

The game's night sky is a small texture whose stars are one texel each. This puts NASA's star map in its place, with four times the pixels: the real sky's stars as fine points, and the glow of the Milky Way. Scandinavia, Kola and Quebec have a photo for a night sky, with the northern lights in it. There the same stars are painted into the photo.

WHAT YOU WILL NOTICE

A night sky full of fine stars, with the Milky Way across it.

WHAT IT COSTS

Video memory for the larger sky textures. The installer is 65 MB larger for the set.

GOOD TO KNOW

It changes boot.pak, together with Photo grade and Sharper smoke and particles, which it keeps, and it sets the levels its photo skies need in initial.pak. The new stars are finer than the game's and come out dim at the game's own level. Brighter stars is set for them.

CREDITS

The star map is made from Deep Star Maps 2020. Credit: NASA/Goddard Space Flight Center Scientific Visualization Studio. Gaia DR2: ESA/Gaia/DPAC.

Skies

Atmosphere · on by default · in depth: Night sky, brighter stars, skies

The painted skies of every region made anew from photographs of real skies: 49 of the game's sky domes, cloud bands and night layers, each at its own size and brightness, the photograph's sun taken out so the game's sun is the only one. The aurora is green, as a real one is. The clouds still drift.

WHAT IT DOES

A region's sky is a stack of layers: a painted dome of the whole upper sky, cloud bands that drift over it, the stars, the sun. This replaces 49 of the game's sky domes and cloud bands, across every region, with ones made from photographs of real skies, the Milky Way on two night layers with the real one from NASA's star map, and the aurora's blue and violet with a real aurora's green. Each keeps the size of the texture it replaces and the game's own brightness for it. The photographs' own sun is taken out, so the game's sun disc is the only sun; the cloud bands are cut out of the photographs with their own transparency.

WHAT YOU WILL NOTICE

Real clouds and skies by day, still drifting; at dusk, dawn and night the game's tint over them.

WHAT IT COSTS

Nothing in the game. The textures come as a download of their own, the sky pack (SnowRunnerNextGen_Skies.zip, about 100 MB, 300 MB unzipped), unzipped into this program's folder; without it Apply leaves the skies out and says so.

GOOD TO KNOW

The rainbow, the comet, the sun's and the moon's discs and glows stay the game's. It changes boot.pak, together with Photo grade, Sharper smoke and particles, Night sky and Reflection cubes, which it keeps. The skies are made with engine\tools\make_cube.js and make_clouds.js from 23 CC0 panoramas by Poly Haven, the Milky Way from NASA's Deep Star Maps 2020.

Brighter stars

Atmosphere · on by default · in depth: Night sky, brighter stars, skies

The stars of the night sky drawn brighter, so they stand out against the dark.

Row	Values (the default in bold)
Brightness	**3x** · 2x

WHAT IT DOES

The night sky draws its stars as one layer, at a low level set per region. This raises that level at night in every region where the stars are their own layer, so they stand out against the dark sky. Dusk and dawn keep the game's level.

WHAT YOU WILL NOTICE

A night sky with clearly visible stars.

WHAT IT COSTS

None: it changes how bright the game draws a layer it draws anyway.

OPTIONS

Brightness: three or two times the game's level.

GOOD TO KNOW

It changes initial.pak, together with Grass reach and Fill light, which it keeps. Three times is the level set for Night sky's star map, whose fine stars come out dim at the game's own level. With the game's own star texture the stars simply come out that much brighter. The regions whose night sky is a photo are not changed by this: with Night sky their photos carry the stars at a fixed level.

Tonemap

Image · on by default · in depth: Tonemap, bloom and photo grade

Stronger contrast and a lighter bloom.

WHAT IT DOES

Tonemapping turns the scene's light into the colours of your screen. This uses a curve with more contrast than the game's own. Mid-grey stays where it was, shade gets deeper and keeps its detail, and bright parts get brighter and roll off without a hard clip. The bloom is turned down to 60 %.

WHAT YOU WILL NOTICE

Deeper shadows and richer colour, less of a washed-out look.

WHAT IT COSTS

None to speak of.

Bloom soft cut

Image · on by default · in depth: Tonemap, bloom and photo grade

No haze from bright snow and sky.

WHAT IT DOES

Bloom makes bright things glow. The game's bloom starts early, so bright snow and sky spread a haze over the picture. This gives it a soft threshold: only really bright things glow.

WHAT YOU WILL NOTICE

Clear snowy views without the milky haze; lamps and the sun still glow.

WHAT IT COSTS

None to speak of.

Photo grade

Image · on by default · in depth: Tonemap, bloom and photo grade

Colour closer to a photo: greens, autumn leaves and the violet haze less loud, whites a little lower, no lens dirt over the sky. Built into the game's own colour grading.

Row	Values (the default in bold)
Strength	**100 %** · 50 %

WHAT IT DOES

The game finishes every picture with a colour grade for the time of day. This builds a photographic grade into it: greens, autumn leaves and the violet haze lose some of their loudness, and whites land a little lower instead of glaring. How much of each colour goes was measured against DLSS 5 shots of the same scenes. Greys stay grey, and the game's own grade stays underneath. It also takes out the game's lens dirt: soft discs the camera adds over bright sky and clouds.

WHAT YOU WILL NOTICE

Foliage and hills in the calmer colours of a photo, a haze that reads grey-blue rather than lavender, less glare on white paint and snow, and no blurry discs over the clouds.

WHAT IT COSTS

None: the game grades the picture anyway.

OPTIONS

Strength: 100 % is the measured grade, 50 % half of it.

GOOD TO KNOW

It changes the four colour tables that the day, dusk and dawn light use (a few night scenes share them), inside boot.pak, where Sharper smoke and particles and Night sky go too; the garage, the map and the darkest night table keep theirs. Best together with Fill light, which does the shade: this one does the colour. When another mod has added its files to boot.pak (a texture pack, say), Apply asks before it builds on them, and they stay in (see How it works, Other mods).

Scenery

Scenery detail

Scenery · on by default · in depth: Scenery detail and grass reach

Scenery keeps its full detail farther out, so less pops in while you drive. All meshes makes shared.pak larger than 2 GB. Shown far away: plants (trees, bushes, rocks, twigs), or everything, no longer vanish at a set distance. Small shadows: what casts none, or loses it a few metres away, gets one.

Row	Values (the default in bold)
Set	nature · **all meshes**
Shown far away	as the game · plants · **everything**
Small shadows	off · plants · **everything**

WHAT IT DOES

Rocks, trees and bushes switch to simpler models close to you (some rocks at 10 metres), and the switch shows as shapes and shadows that flip while you drive. This moves the first switch out to at least 40 metres and the later ones to 70, 110 and 160.

WHAT YOU WILL NOTICE

Far less popping on rocks and trees.

WHAT IT COSTS

More detailed models drawn farther out: some frame rate in dense forest. Each of the two extras adds to that: more plants drawn far away, more shadows drawn.

OPTIONS

Set: nature changes rocks, trees and bushes (351 models). All meshes changes every model that has detail steps (1626), which makes shared.pak larger than 2 GB; the game loads it fine. Shown far away: the game stops drawing most things beyond a distance of their own (small rocks and twigs at 55 to 80 metres, trees and buildings farther), which shows as things appearing ahead of you. Plants takes that distance out of every plant (314 models: trees, bushes, rocks, twigs, roots), everything out of every model that has one (3468: signs, fences, pipes, houses and the rest as well). Small shadows: some models cast no shadow at all or lose it 25 to 50 metres away. Plants gives each of those plants its shadow at any distance (163 models), everything does the same for all 395.

GOOD TO KNOW

A game update or a Steam file check puts the original shared.pak back. Apply again afterwards.

Grass reach

Scenery · on by default · in depth: Scenery detail and grass reach

Grass drawn farther out. Costs frame rate in grassy areas.

Row	Values (the default in bold)
Reach	3x · **2x**

WHAT IT DOES

Most grass fades out about 30 metres from the camera. This draws it out to 60 metres, and it still grows in as it comes nearer.

WHAT IT COSTS

Frame rate in grassy areas: grass covers up to four times the ground, and at 3x it stays thick farther out than at 2x.

OPTIONS

Reach: 3x or 2x the game's fade distances, each step at most 50, 55 and 60 metres.

GOOD TO KNOW

The game draws grass only to about 64 metres from the camera. Set farther, the grass at the edge appears whole at once instead of growing in, so no step goes past 60 metres; a step the game itself sets farther stays. It changes initial.pak, a file that other mods change too.

Interface

Next Gen logo

Interface · on by default · in depth: Next Gen logo and the start without the intro

NEXT GEN under the game's logo on the title screen, the first loading screen, the main menu, the pause menu and the map.

Row	Values (the default in bold)
Start-up	the game's intro · **Next Gen splash only**

WHAT IT DOES

Adds NEXT GEN under the game's logo wherever the interface shows it: on the title screen, the first loading screen, the main menu, the pause menu and the map.

WHAT YOU WILL NOTICE

The mod's name under the SnowRunner logo, so you can tell at a glance that it is installed.

WHAT IT COSTS

Nothing in the game. The first Apply keeps a copy of gfx.pak, several hundred MB, so that Restore can put it back.

OPTIONS

Start-up: the game's intro (its warning and the publisher logos, about 18 s), or the Next Gen splash only: the game goes straight to its first loading screen. Measured on 2026-10-17 with a warm disk cache: the title screen came 41 s after start with the intro and 24 to 25 s without. The splash only is the default.

GOOD TO KNOW

It changes gfx.pak, the file of the interface. The start-up row changes the game's start-up movie in the same file: two small edits to its script, and a game version whose movie differs keeps its intro (Apply says so). The title screen, the first loading screen and the main menu have a picture of their own for the logo, and those are replaced whole. On the pause menu and the map the logo shares a sheet with other pictures: there only the logo changes, so an interface mod's sheet keeps everything else, and a sheet with another layout is left as it is.

Part 4. How it works, in depth

This part explains how each card works: the technique from first principles, the formulas and numbers it uses, the passes and buffers it reads and writes, what the patch changes in the game and why, what it costs and where it falls short. The numbers are the ones in the code; where a knob of SnowRunnerShadows.ini sets one, its name is given.

How Next Gen gets into the game

SnowRunner draws with Direct3D 11. Every picture on screen comes from small programs on the graphics card, shaders, which the game keeps precompiled in one file, **shader.pak**. Next Gen changes the game in four ways, and every card in this part uses one or more of them.

1. Whole shaders replaced

Some of the game's shaders are swapped for ours, written in HLSL and compiled with Microsoft's compiler (fxc) to the same bytecode format, DXBC. The ambient occlusion pass (two shaders, hashes 0xEA2414F8 and 0xA3716E2B) becomes GTAO, the fog shader (0x871EF8CC) gets jitter and a phase function, the tonemap (0x221304E2) and the bloom (0x52879E18) get new curves. Inside shader.pak the shaders sit in one entry, *shadercachedx11.sdc*: a 12 byte header (a date stamp and two sizes) and one zlib stream. Inflated it is a list of includes, a count of programs, a count of blobs, then every blob (a DXBC shader, prefixed with its size) and a table that ties programs to blobs by index. The post-process shaders sit in a second cache, *common_pc_sm50_hdr.sdc*, in the same form. A replaced blob may change size: the tool rewrites the stream and the table and compresses it again.

2. Helpers spliced into the game's own shaders

Most cards keep the game's shader and add to it. The game has thousands of material shaders, one per kind of surface (3229 for vehicles and props alone), and replacing them all would mean rewriting the game's materials. Instead a patch tool finds one instruction in each, an **anchor**, and puts a call to our helper there. For the reflections the anchor is the material's sample of the sky cubemap:

> sample_l r4.xyz, r5.xyz, t86.xyzw, s8, r3.w // cubemap t86 at mip $\mathrm{r3.w} = \sqrt{\mathrm{roughness}} \times 8 + 0.5$

The helper takes that sample's place. It gets what the sample got (the direction and the mip level) and what it needs besides (the pixel's world position), and its result goes where the sample's went, so everything the game does after it (the reflectivity, the Fresnel term, the scale g_fReflCubeGGXScale) still applies. The helper is compiled once from HLSL; its instructions are copied into each target shader with their registers renamed to free ones.

Splicing compiled code is surgery on a container. A DXBC file holds chunks (the program in SHEX, its inputs and outputs, statistics) and a checksum. After inserting instructions the tool (dxbc_patch.js) repairs everything that depends on the layout: the container and chunk sizes, the chunk offsets, the program length, the number of temporary registers (dcl_temps), the instruction counters in the statistics chunk, and the checksum, an MD5 variant with a non-standard final block, verified against the game's own shaders. A target is left alone unless the patch can prove it fits: exactly one anchor, the inputs it needs declared, nothing after the anchor reading a register only the anchor wrote. Every patch tool works on a stock shader or on one another card already changed, because the cards touch different instructions.

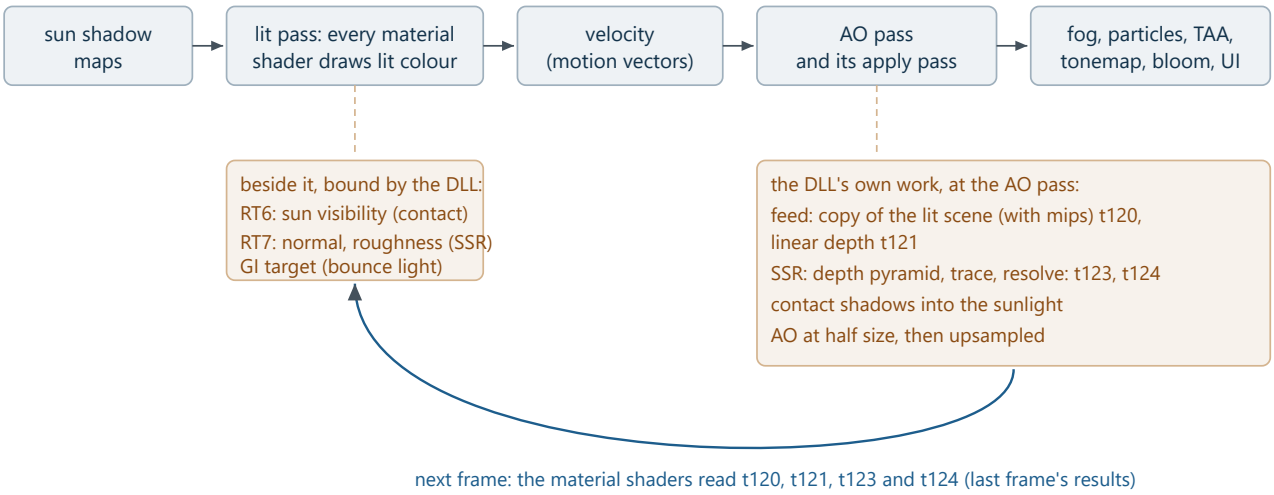
3. Numbers changed in place

Some cards only change a constant: the headlight glare cap (32 becomes 6), the decode of a light map, a frame number in the start-up movie. These patches keep the size of everything they touch.

4. SnowRunner Shadows, the DLL

Some techniques need data no single shader has, such as the whole lit frame or a depth pyramid. The game loads a file named hid.dll from its own folder before the system's copy, so Next Gen places

SnowRunner Shadows there; it passes every call on to the real hid.dll (or to another mod's, renamed hid_chain.dll) and hooks the Direct3D 11 device context. It watches which shaders the game binds, recognises the passes it needs by their shaders, and adds its own work between them. It never writes into DXGI's shared method tables, which other mods and Steam's overlay hook too: its diagnostic options (the frame log, GPU timers, the profiles) take the game's swap chain and factory, and each gets a copy of its method table of its own with the hook in it. Steam's client of 2026-10-21 hooks over hooks it finds in DXGI's shared Present, and a hook there that had saved Steam's as its real function then called itself back until the stack ran out (found in a player's crash dump with another mod). Its own Direct3D 11 hooks remember, for each method table they sit in, the functions they replaced and eight slots they never touch; if another program later gives the device or a context a private copy of that table (the hooks come along), the copy is known by those eight slots and takes the same functions, instead of the next call failing.



One frame as Next Gen sees it. The game's passes are on top; what Next Gen adds is below them. Results of the DLL's passes reach the material shaders one frame later, so every reader reprojects into the previous frame.

The DLL's parts that the cards rely on:

What	Where it lives	Who reads it
The feed: the lit scene of the last frame with a mip chain, and its linear depth	t120 and t121, bound to every shader that declares them	object reflections, water, wet ground, bounce light
Render target 7: world normal × 0.5 + 0.5 and perceptual roughness (RGBA8)	bound beside the lit pass, written by the material shaders (card SnowRunner Shadows with Object reflections)	the reflection pass
The reflection pass's result: radiance × confidence and confidence (RGBA16F), and object motion	t123, t124	the material shaders, next frame
Render target 6: the sun's visibility each receiver found	bound beside the lit pass	the contact shadow pass
A second AO target: light bounced from nearby surfaces	written by the AO pass	Bounce light

Without the DLL every spliced shader still runs: a helper that finds its textures unbound returns exactly what the game's own instruction would have returned. F8 in the game switches all of Next Gen's shader work off and on, so a look can be compared from one spot. For that the DLL keeps a table of the game's original of

every changed shader (SnowRunnerShadows.stock, about 5,600 of them) and draws the original in place of the changed one while F8 holds (F11 the same for the AO pass alone). The originals are built the first time F8 or F11 asks for them, each shader once, and kept: the first press takes a moment, and a start builds none. Until October 2026 they were built while the game started, about 5,600 shader builds before the first frame that no comparison needed.

The same switches are buttons in the NEXT GEN overlay, the menu the Next Gen mods share (F2 opens it; NextGen.ini beside the game's exe can name another key). The DLL exports the list (NgGraphicsGetApi, version 1): every switch whose feature the ini turned on, with a name, a tip, its state and a way to set it, and a few status lines (device hooked, shadow map size, stock shaders, ReShade). The overlay finds the DLL by its full path in the game's folder and draws the Graphics tab from that list on the game's present thread. A button and its key flip one flag, so they never disagree; a change lasts until the game closes, and SnowRunnerShadows.ini sets the start. While a NEXT GEN menu has the Graphics tab the keys F4 to F11 are silent, because other mods use some of the same keys (KeysWithMenu=1 in the ini keeps them); with Next Gen Graphics alone they work as before. Every Next Gen mod writes the game's import slots under one lock (Local\NextGen.Slots.<pid>) with a compare-exchange that keeps what was there as the original, so two mods hooking D3D11CreateDevice chain in either order.

Sun shadows: resolution, edges, softness, seams

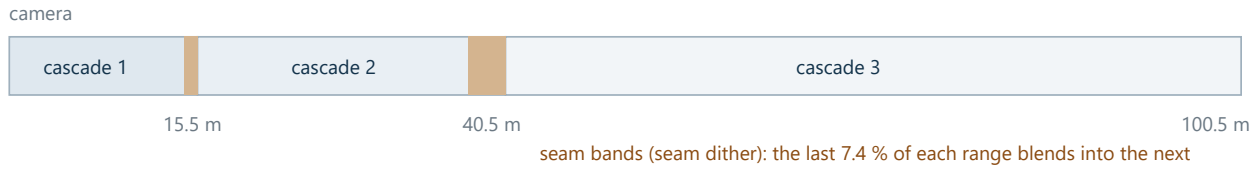
Cards: SnowRunner Shadows (Resolution, Self-shadow fix), Shadow edges, Blocker search. Four changes to how the game's sun shadow is drawn and filtered.

Shadow mapping in one paragraph

Before the frame, the game draws the scene from the sun into a depth texture: each texel holds the distance from the sun to the nearest surface along that line. When a pixel of the frame is shaded, its position is projected into that texture; if the stored distance d_{map} is smaller than the pixel's own distance d_{recv}, something lies between it and the sun:

> $$\text{lit} = [\, d_{recv} - \text{bias} \le d_{map}(u, v) \,]$$

One such test gives a hard edge with the texture's texels as stairs. Filters average many tests around the pixel (percentage-closer filtering, PCF), which softens the edge. SnowRunner keeps three cascades, one texture each for the near, middle and far range, side by side in one depth texture of 4608 × 1536 at its top setting (1536 × 1536 per cascade). A pixel uses the first cascade whose split distance lies beyond it: 15.5 m, 40.5 m, 100.5 m from the camera.



The three cascades along the view depth. Each next cascade's texels cover 2 to 2.5 times more ground.

Resolution: a bigger shadow texture (SnowRunner Shadows, row Resolution)

No game file sets the shadow texture's size: it comes from the game's executable. The DLL recognises the texture when the game creates it (a depth texture three times as wide as high), creates it Factor times larger, and while it is the bound depth target scales every viewport and scissor rectangle the game sets by the same factor. The shaders read it with normalised coordinates (cascade offsets 0, 1/3, 2/3), so they need no change. Factor 3.5 is the largest that fits: 4608 × 3.5 = 16128, just under Direct3D 11's limit of 16384. A texel then covers $1/\text{Factor}^2$ of the ground it did. Cost: at Factor 3.5 about 7 ms more per frame at 3840 × 2160. The card's default is 1x, because the rebuilt edges below give straight shadows at the game's own size.

Self-shadow fix (row Self-shadow fix)

To keep a surface from shadowing itself (shadow acne), the game draws the shadow texture with a slope-scaled depth bias: each primitive's depth is pushed back in proportion to its depth slope across *one texel*. A texture Factor times finer has texels Factor times smaller, so the same setting gives Factor times less bias, while the filters still reach as far as before. Surfaces at a grazing angle to the sun then start to shadow themselves, differently in each cascade (the dark side of a boulder flips at the 15 m border at dusk). With the fix on, the DLL swaps the game's rasterizer states, while the scaled texture is drawn, for copies with the slope-scaled bias multiplied by Factor. Every primitive gets exactly the bias it gets at the original size.

The game's filter, and why it shows stairs

The game's high quality shadow path (1269 material shaders: trucks, objects, terrain) runs three steps: pick the cascade, search for blockers to estimate how soft the edge should be (a penumbra size), then filter with 16 comparison taps at fixed Poisson-disc positions, stretched by the penumbra. Two things show:

- At Factor 1 to 2 one texel covers 2 to 16 screen pixels near the camera, and the game's contact penumbra is about half a texel. Any filter narrow enough to keep contact shadows crisp shows the texel staircase.
- When the penumbra spans more texels than 16 taps cover, every tap adds its own stair-stepped copy of the edge.

Shadow edges, the crisp choice: a tent over a grid

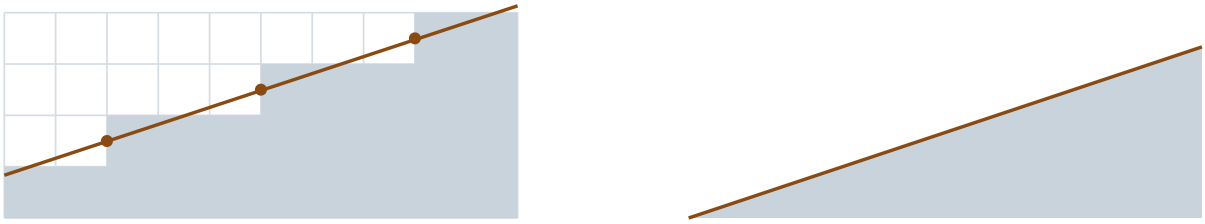
The first replacement filter (hq_grid) keeps the game's cascade choice, blocker search and penumbra, and replaces only the 16 taps. It takes a tent (a linear falloff, half width H = 1.6 kernel units, matching the spread of the stock taps) over a grid of bilinear comparison taps whose spacing is measured in real texels, with as many taps per axis as the width needs (2 to 8). A bilinear comparison tap already returns the lit fraction of four texels, weighted by position, so the grid integrates the tent over the texels:

$$\text{lit} = \Sigma_{i,j}\, w_i\, w_j \cdot \text{cmp}(u_i, v_j), \quad w_i = \max(0, 1 - |x_i| / H) \,/\, \Sigma_k \max(0, 1 - |x_k| / H)$$

No noise is used, so it needs no TAA to look clean.

Shadow edges, the default: revectorization

The default filter (hq_revec, written from Macedo and Apolinário, "Revectorization-Based Shadow Mapping", GI 2016, and Bondarev, "Shadow Map Silhouette Revectorization", I3D 2014) rebuilds the straight edge that the texels only sample.



texel tests (grey: shadow) and the line fitted through the steps' middles

the edge drawn as the line, softened by the tent across it

Revectorization: from stairs of texel tests to the straight edge they sample.

1. Four GatherCmp instructions read the depth tests of the 4 × 4 texels around the pixel. All equal: the pixel is plainly lit or shadowed, done (most pixels).
2. The edge's main direction: more changes between rows than between columns means a mostly horizontal edge; the rest works in that frame. Then the change nearest the pixel in its own column, between rows R and R + 1.
3. The edge is followed both ways, two columns per GatherCmp, up to 8 texels each way and inside this cascade. Where rows R and R + 1 stop differing, the texel beyond tells whether the edge stepped one row up or down (a step: the edge crosses the step's middle, so the step joins the fit and the follow goes on) or stopped (a corner, or a pattern that is neither). At most 4 steps each way join the fit.
4. The edge as a line: the least-squares line through the steps' middles (x_k, y_k), each weighted by its distance from the pixel, $w_k = \exp(-x_k^2 / 2\sigma^2)$ with $\sigma = 4$ texels:

   > $\text{minimise } \Sigma_k\, w_k\, (y_k - a - b\, x_k)^2 \;\Rightarrow\; b = (\Sigma w\, \Sigma wxy - \Sigma wx\, \Sigma wy) \,/\, (\Sigma w\, \Sigma wx^2 - (\Sigma wx)^2),\;\; a = (\Sigma wy - b\, \Sigma wx) \,/\, \Sigma w$

   The line is used only if the middles lie within 0.3 texels RMS of it. A single step fades back to the run within the follow's reach (an L shape, placed as morphological antialiasing places it). When the first steps both ways go the same way (a U shape: a bump, the top of a round shape) the line bends in the middle of the pixel's run. With no step in reach, the edge lies flat, halfway between the rows.
5. The lit fraction is the tent's profile across that line: a straight edge as soft as the game asked for, without the texel steps. For a signed distance s from the line, in texels, and tent half width H:

   > $\text{lit}(s) = ½ + s/H - s|s| \,/\, (2H^2) \quad \text{for } |s| \le H, \text{ 0 or 1 beyond}$

   This is the integral of the normalised tent $(1 - |x|/H)/H$ up to s.

Wide penumbrae, from a tent half width of 1.25 texels, take the tent over the texels: their steps are blurred already, and a 4 × 4 look cannot see an edge that far. From half that width the two are blended. Texture-like patterns (leaves, grass: many changes within the 4 × 4) take the tent as well.

Blocker search: how soft each edge is

Shadows are sharp where an object touches the ground and soften with distance. The game estimates this like PCSS: it averages the depths of the blockers found around the pixel, d_{avg}, and turns the ratio

> $p = (d_{recv} - d_{avg}) \,/\, d_{avg}$

into the penumbra size. The game's search reads 16 texels at fixed offsets; as they cross a blocker's edge one by one, the size jumps in visible steps. Next Gen's search (hq_blocker) reads 64 texels on a 4 × 4 grid of Gather fetches over the same area (the outer cells centred at 0.63 of the search radius, where the stock offsets reach), with the stock rule that every texel counts as $\min(\text{depth}, d_{recv} - 0.001)$, and hands the same ratio back. The game turns it into the penumbra exactly as before; it only changes smoothly now.

The seam dither

At each split distance a shadow jumps from one cascade's texels and kernel to the next one's (texels 2 to 2.5 times larger, a kernel narrower by a factor e): a line across the ground 15.5 m from the camera. The seam patch replaces the cascade choice's compare. The pixel's depth z is raised by up to 8 % of itself with noise n per pixel and frame before the compare:

> $z' = z\,(1 + 0.08\,n), \quad n \in [0, 1)$ (interleaved gradient noise, shifted by the game's time)

So over the last 0.08 / 1.08 = 7.4 % of each cascade's range (14.4 to 15.5 m, 37.5 to 40.5 m, 93.1 to 100.5 m) a growing share of pixels takes the next cascade, and the game's TAA averages the two. Pixels never take a nearer cascade.

Limits

- Revectorization assumes straight edges. Detail smaller than a texel (thin branches, wires) cannot be rebuilt from texels that never saw it; there the filter keeps the tent.
- The seam dither relies on TAA; with TAA off the band shows as a dotted transition.
- Resolution costs time on the shadow pass and memory: Factor 2 is four times the texels.

Contact shadows

Card: Contact shadows. Fine sun shadows traced on screen where things touch or overlap: under a tyre, where a log lies on another, the creases of a cab.

Why the shadow map misses them

A shadow texel covers centimetres to decimetres of ground, and the depth bias that prevents acne pushes the comparison back further. Shadows smaller than that, such as the thin dark line where a wheel meets mud, fall between texels. The camera's own depth buffer has a pixel's resolution, so tracing towards the sun in screen space finds them.

The trace (Bend Studio's screen-space shadows)

The pass is Bend Studio's screen-space shadow method (from Days Gone; Apache 2.0, a modified copy ships in the DLL). The sun is projected to a point on screen. Every pixel lies on a line through that point, so the screen is cut into lines radiating from it, and one wavefront of 64 threads walks 64 neighbouring pixels of one line together, sharing the depths they read. For each pixel, 60 depth samples are read towards the sun along its line: a contact shadow can be up to 60 pixels long. A sample occludes when it lies in front of the ray from the pixel towards the sun, every sample standing for a surface of a set thickness (ContactThickness, 0.005 in the pass's depth units); a contrast of 4 sharpens the result. The first 4 samples count as hard shadow, the last 8 fade out, and the result is a mask m from 1 (lit) to 0 (shadowed). The pass reads the game's linear depth as 1/z and skips pixels beyond 50 m (ContactMaxDepth).

Into the picture: only the sunlight

A screen-space shadow must not darken light that is not sunlight. So the patch (module contact) makes each of the 1269 sun-receiving material shaders write the sun visibility it found (the shadow map's lit fraction after the game's floor, g_fShadowAtten) to render target 6. After the lit pass the DLL draws a full-screen pass that multiplies the colour by

> $$k = 1 - (1 - m) \cdot \text{share} \cdot \text{strength} \cdot \text{fade}$$

> $$\text{share} = \text{sun} / (\text{sun} + \text{sky}), \quad \text{sun} = L(C_{\text{sun}}) \cdot m_{\text{pbr}} \cdot \max(0, N \cdot L) \cdot v, \quad \text{sky} = L(\text{the three sky colours blended by } N_y)$$

with v the visibility from render target 6, C_{sun} the game's sun colour and m_{pbr} its PBR multiplier, L() the luminance, N the shading normal (render target 7 where a material wrote it, else rebuilt from the depth buffer). A pixel the sun lights fully darkens to about what a shadow-map shadow would give there; one already in shadow or facing away is not touched. With distance the effect fades: fade = 1 − smoothstep(30 m, 50 m, z). Far away a contact shadow is smaller than a pixel, and ground seen at a flat angle starts to shadow itself in screen space.

Limits

- Screen space only: an occluder outside the picture, or hidden behind something nearer, casts nothing.
- The thickness is one number for everything: very thin things (wires) may cast too long a shadow behind them.
- F4 switches the pass off and on.

Headlight shadows

Card: Headlight shadows, on by default. The truck's headlights cast shadows: a post, a tree or a rock in the beam leaves the ground behind it dark.

How the game lights with its headlights

The headlight is a projected spot light. CB_GLOBAL_SCENE holds its position (g_vHeadLightPos), direction, cone, distance falloff and colour, and the matrix that projects a world position into the lamp's light pattern (g_tmHeadLightViewProj). Each of the about 5100 lit pixel shaders that take the headlight measures the pixel from the lamp, tests the reach and the cone, fades the light out within 3 to 6 m of the lamp and applies the distance falloff and the colour. The objects' shaders (about 2600) also sample the pattern texture g_txHeadLightMap once; the terrain's (about 2300) work out the intensity alone and multiply the lamp's colour by it. Nothing in either chain asks whether anything stands between the lamp and the pixel: the game has no shadow map for its headlights.

The walk to the lamps

The patch (module headshadow) splices a helper in right after the pattern sample and multiplies the sample by what the helper returns, or, in the terrain's shaders, right before the lamp's colour multiply and multiplies the intensity by it: v, the share of the light that arrives. For a pixel at P and a lamp at L the helper walks from 0.12 m off the surface towards L and stops 2.5 m short of the lamp (the truck's own body stands there, and the game has faded its light out that close anyway). Both ends are projected with the previous frame's view-projection (cb1 c10..c13), as the depth they are compared with is that frame's, and the walk takes one step per 6 pixels of its length on the screen, at least 24 and at most 64, evenly spaced, so a pole wider than the gap between two steps is not stepped over (a thinner one can be: see the limits). With a fixed 24 steps a long walk jumped over a fence's rail a few pixels thin, and its shadow came out as a comb of bars, the worse the wider the field of view (seen in the game, 2026-10-27). A step's distance from the camera is interpolated as 1/w:

> $$w(f) = 1 \,/\, \mathrm{lerp}(1/w_0, 1/w_1, f), \quad f = (i + j) \,/\, n$$

with j an offset per pixel (interleaved gradient noise, Jimenez 2014) that the game's TAA averages out. At each step the scene's linear depth z on that pixel (SnowRunner Shadows' copy of the previous frame's depth, t121, taken at the AO pass, which comes after the lit pass) is compared with w:

> $$d = w - z - (0.04 + 0.006\,w), \quad b = \mathrm{saturate}(d \,/\, 0.15) \cdot [d < 1.5 + 0.03\,w] \cdot e$$

d > 0 means the scene stands in front of the step. More than the thickness (1.5 m plus 3 % of the distance) in front, the ray passed behind the caster, not through it. e fades the test out over the last 3 % of the screen at its border. The lamp's visibility is 1 − max b over the steps.

What blocks is placed in the world: on the camera's line through the step, at the scene's depth there, O = eye + (S − eye) · z / w, with S the step's own position. An obstacle less than 0.3 m in front of the lamps' plane does not count: nothing behind a lamp can stand between it and a lit point. Without that rule the truck's own cab and roof rack, seen from a camera above and behind, lay within the thickness in front of the road and darkened the beam next to the truck (seen in the game, 2026-10-23).

A low headlight's rays leave the ground at a flat angle: for the first metre they run a few centimetres above it. There snow's small lumps blocked them, at a different point for each pixel's offset: a fine grain over the

whole lit snow, with stripes on some slopes (seen in the game, 2026-10-26). Each ray now starts 10 cm above the surface, straight up (HS_LIFT), so lumps lower than that do not block. In the test scene with 12 cm drifts and 4 cm roughness and no casters, the share of the beam read as blocked is 0.01 % from behind the truck and 0.02 % from the chase camera, against 5.1 % and 5.2 % without the lift. A second change tried with it, letting the farther of the depths one pixel row above and below count where a step seemed blocked (HS_ROWS, still in the code), cleared the rest in the test scene but cut thin rails' shadows into combs, so it is off.

Nor does an obstacle within 2.5 m of a lamp (HS_BODY, the same as the walk's stop). The walk never comes that close, so what the camera sees there is the truck's own front (a bumper, a winch, a bull bar), found only where it stood in front of a step further out. From a low view beside the bumper it blocked every ray passing behind it: a hard-edged dark quad on the road, its upper edge level with the bumper's (seen in the game, 2026-10-24). In the test scene's view beside the bumper the rule lights 14,272 of 175,948 beam pixels that were blocked, from beside the truck 1,585, and changes nothing from behind or from the chase camera; the posts, the guardrail, the wall and the pole cast as before. In the game the same rule also took away hard-edged dark patches on the road ahead of a truck climbing a hill, which looked like the road's own mesh casting (seen in the game, 2026-10-24).

The first builds projected the steps with this frame's camera. With the camera still that is the same; with the camera moving (shaking on a rough road) each step met whatever the previous frame had at that spot on the screen, and the road ahead went dark until the camera settled (seen in the game, 2026-10-23). With the previous frame's camera a step meets the depth that camera saw there; O is still placed from this frame's eye, a frame's movement off, which moves it by that movement times (1 − z / w), a few centimetres.

The walk starts on the surface the camera saw. The pixel is projected with the previous frame's camera too and compared with the depth there; one up to 3 m (plus 25 % of the distance) behind or in front of it is first moved along this frame's camera ray onto that surface. A road overlay or a decal drawn over the ground, parallax-mapped asphalt or a depth bias leave the point a shader lights a few centimetres under the depth the camera saw, and seen at a grazing angle those centimetres put the first steps tens of centimetres behind the road, which then blocked them all: a hard-edged dark patch the size of the overlay, gone from a steep view (seen in the game, 2026-10-23). In the test scene a patch of road whose depth lies 5 cm above its shaded pixels darkened 926 of 176,000 beam pixels from a low camera in front of the truck and none from above; with the move, 25 (3 without the patch). At 15 cm, 14,400 against 2,180: the raised depth's far edge then shades the ground just behind it, as a step that high would.

A pixel further off than that keeps the game's own light, unshadowed. On some surfaces the position the game hands the shader lies metres from the depth the camera saw there, in front and behind alike: on a snow bank ahead of a truck, seen at night with the headlights on, the walk from that position was blocked over the whole bank, a large dark patch with straight edges, and moving the point onto the depth, even with no limit on the distance, did not help (seen in the game, 2026-10-27, with a test build that showed each pixel's agreement instead of the shadow). Why the position is that far off is not known yet. An edge, or something the last frame did not see there, falls under the same rule: its shadow is lost for a frame.

A truck's two headlights stand about 1.4 m apart, and the game hands its shaders one position between them. The helper walks to a point 0.7 m to each side, level and across the lamp's direction, and averages the two, so a pixel that only one lamp reaches gets half the light: a shadow's edge comes out as a half-lit band, the wider the nearer its caster stands to the truck. That is at most 128 depth reads for a pixel in the beam, and none outside it, since the splice sits inside the game's own test of the reach and the cone.

Which shaders

The helper needs the pixel's world position as the shader measured it from the lamp: an input from the vertex shader, or a temporary register, taken only where no instruction overwrites it before the helper runs.

5123 of the 5143 shaders take it; the other 20 (particles whose position comes without its third component, or none at all) keep the game's light. Each patched shader gets its camera constants declared up to the previous frame's view-projection (c13), and t121 declared. The grass's and the bushes' shaders (267) take the headlight but declare no camera constants at all; the patch declares them (b1, which every pixel shader of the game that uses it binds to CB_GLOBAL_CAMERA). Until 2026-10-23 they were left out, and a post or a guardrail standing in a verge cast nothing, while a road cone on the asphalt did (seen in the game): their shadows fall on grass. The module is patched last, on top of what the other modules made of each shader. All 5123 pass the build's input check and are created by Direct3D 11, on WARP and by the GPU's driver (checked offline, 2026-10-23). The first build reached the objects alone: in the game a post's shadow fell on the pole behind it but not on the ground, which is how the terrain's own way of lighting was found.

Limits

- Screen space only: a caster outside the picture, or hidden behind something nearer the camera, casts nothing, so a shadow can change as the camera turns.
- Nothing within 2.5 m of the lamps casts: the truck's own bumper, winch and bull bar do not, and neither does a post that close.
- Thin casters can still be missed: where the walk is longer than 384 pixels on the screen, its 64 steps lie further apart than a thin pole is wide. A walk over the pyramid of the nearest depths (t125, the reflection pass's), which cannot step over anything, is the next step if the game still shows such gaps now that the grass takes the shadows (see Which shaders).
- A raised overlay's far edge can shade a short band of the ground behind it, as a step that high would (see the start of the walk).
- Grass is drawn several times over where it is thick, and each layer in the beam now walks to the lamps: there the helper costs more than the times below.
- The game has slots for four headlights; only the first is shadowed.
- Without SnowRunner Shadows, or from 250 ms after its copies stop (a menu), t121 is unbound and the light is the game's own.
- Seen working in the game once (a post's shadow on the pole behind it, against F8); not tuned yet.

Testing without the game

An offline test scene (test\headshadow_scene) ray-casts a synthetic night scene per pixel: ground, a truck body at the lamps, posts, a pole, a concrete block, a low block 1.6 m ahead of the bumper, a wall, a guardrail on posts along the left of the road and a thin marker post; OVERLAY_H raises the depth of a patch of road above its shaded pixels, as an overlay does. It writes the scene's linear depth as the DLL's t121 holds it, fills the camera and headlight constants as the game does, and shades the scene with the helper itself, from beside the truck, behind it (the chase camera), above, low in front of it beside the bumper and high behind it further off (the last two as in the game's shots of the dark patch). Each setting can be overridden there. Seen in it: the shadows fall away from the lamps, doubled up close; with the chase camera the truck's own cab blocks nothing; at 1.5 m instead of 2.5 m the low block casts too and the truck still blocks nothing, which a real truck's bull bar or winch may not share.

Its measure.ps1 times the helper alone, against the same pass without it, one sample per pixel as in the game, the fastest of 60 draws on an RTX 4080: at 3840 × 2160, 0.19 ms with 5 % of the screen in the beam (the chase camera), 0.52 to 0.84 ms with 15 to 23 %, 1.2 ms with the camera low behind the lamps and 39 %. With at most 24 steps, as in 1.1.0, the same views take 0.05, 0.26 to 0.35 and 0.54 ms: the longer walks cost about twice as much. In the game the lit shaders run once per surface drawn, so places drawn over several times (grass, foliage) pay more.

Ambient occlusion: GTAO and the wide reach

Cards: GTAO (row: half or full size), Wide occlusion. Shade in corners, under trucks and in wheel arches, where less of the sky reaches.

What occlusion measures

Ambient light comes from the whole sky. A point in a corner sees less of it. The quantity is the visibility V: the share of the hemisphere above the surface that is open, weighted by the cosine to the normal (light from straight above counts most):

$$V = (1/\pi) \int_{\Omega} \text{visible}(\omega)\ (N \cdot \omega)\ d\omega$$

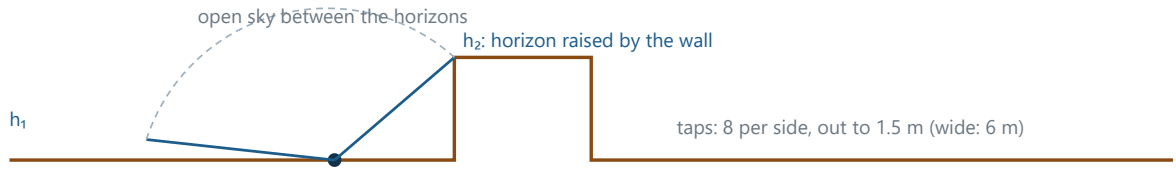
The game's own pass estimates this with 8 depth samples on a fixed kernel whose size is a fixed share of the screen, whatever the distance: a box looks occluded the same whether it is 2 m or 200 m away.

GTAO: integrating horizons

Ground truth ambient occlusion (Jimenez et al. 2016, "Practical Real-Time Strategies for Accurate Indirect Occlusion"; the integration follows Intel's XeGTAO, MIT) splits the hemisphere into slices: planes through the view direction, at screen angles φ. In each slice the open sky is the arc between two horizons, h_1 on one side and h_2 on the other: the steepest angle to any surface within a radius. The integral over that arc has a closed form. With n the angle of the normal projected into the slice and $|n_p|$ the length of that projection:

$$a(h) = \tfrac{1}{4}\ (-\cos(2h - n) + \cos n + 2h \sin n)$$

$$V \approx (1/S)\ \Sigma_{\text{slices}}\ |n_p|\ (a(h_1) + a(h_2)),\quad AO = V^{1.6}$$



One slice: the open arc between the two horizons is the share of the sky the point sees in that direction.

The numbers: 4 slices per pixel, 8 depth taps per side of each slice (64 depth reads), a radius of 1.5 m, capped at 12 % of the pixel's depth so the kernel stays bounded on screen near the camera. The normal is rebuilt from the depth buffer. The slice angles and tap positions are rotated per pixel by interleaved gradient noise; the game's own 3 × 3 blur in its apply pass averages the noise out. The effect fades out between 150 m and 300 m, where fog takes over. The field of view comes from the game's camera every frame. The pass replaces the game's two SSAO shaders and keeps their inputs and outputs: x = visibility, yzw = the centre depth for the blur.

Half size

With the GTAO row at half size (and SnowRunner Shadows' AOHalf = 1) the pass runs on a quarter of the pixels, each half-size pixel snapped to the depth of one full-size pixel it covers, and the DLL upsamples the result weighted by depth, so occlusion does not bleed across object edges.

Wide occlusion

Big shapes (a truck's body, a roof, a tree crown) shade the ground further than 1.5 m. The wide reach continues each search past the near radius to 6 m, with 4 more taps per side, capped at 50 % of the depth near the camera. A far occluder does not count fully: at the near radius it raises the horizon half way (strength 0.5), fading to nothing at 6 m, so the shade under a truck is soft and the crisp contact occlusion near the ground stays as it was. To save time each 2 × 2 quad of pixels shares the far search: each pixel searches one of the four directions far and the four results are shared. That is a quarter of the far reads: 0.9 ms instead of 3.3 ms at 3840 × 2160.

Limits

- Screen space: what is outside the picture or hidden behind something casts no occlusion.
- Thin things in front of a wall make a dark halo on the wall; the depth weighting and the fade keep it small.
- F11 compares GTAO with the game's SSAO, F5 switches between half and full size.

Bounce light

Card: Bounce light (off by default, experimental). Sunlit ground and paint light up what stands next to them, in their colour.

Idea: the horizons already know who blocks the sky

The GTAO search finds, in each direction, which surfaces hide the sky. A surface that hides part of the sky sends its own light instead. Each tap that raises the horizon from h to h' hides a further arc of the sky, and that arc now sees the surface the tap found. Its share is the same cosine-weighted measure GTAO integrates, a(h') − a(h). So the bounce light is

> $$B = s \cdot (1/S) \, \Sigma_{\text{slices, sides}} \, \Sigma_{\text{taps that raise the horizon}} \, |n_p| \, (a(h') - a(h)) \cdot \min(C_{tap}, 64)$$

with C_{tap} the tap's colour in the lit image of this frame (the DLL's copy, t120), clipped at 64 to drop sun glints, and s = 4 the strength. A pixel boxed in by sunlit snow gathers the snow's light; one next to a red truck gathers red. (The derivation is our own, from the GTAO arc integral; the idea is that of horizon-based indirect lighting.)

Keeping a feedback loop stable

The bounce light becomes part of the lit image that next frame gathers from: light can bounce again and again. Two caps keep the loop bounded. The stored value never exceeds 0.9 times the brightest surface it was gathered from. And the material side never adds more than a set multiple of the brightest of its own ambient colours.

Into the materials

The game lights shade with three ambient colours: from above, from the horizon and from below (cPosY, cMidY, cNegY), blended by the normal and multiplied by the surface colour. The patch (gi_ambient) adds the bounce light to all three, so the game's own code carries it on with the real albedo. Because the bounce was measured last frame, each pixel looks it up where it was then: projected with last frame's camera if the world stood still, or at the same screen position if it moved with the camera (the player's truck). Last frame's depth decides which applies; where neither matches (just uncovered), no bounce light this frame. The AO pass stores the bounce divided by the darkening the AO apply pass will give the pixel, so after the apply the bounce keeps its strength.

Limits: it is experimental because in places (bright snow next to dark wood) it comes out too strong, and like every screen-space effect it knows only what is on screen. F10 switches it off and on.

Sky ambient and fill light

Cards: Sky ambient, Fill light. The colour and the amount of the light that fills in shade.

Sky ambient

Each daytime state sets the three ambient colours as one warm grey at three strengths, so shade has no colour variation: under a blue sky it should lean blue, at sunset warm. The patch (sky_ambient, spliced at the start of every material shader that reads those colours) keeps each colour's brightness and takes the hue of the sky in its direction from the reflection cube the shader already binds, at its roughest mip (close to the sky's irradiance): straight up for cPosY, straight down for cNegY, the mean of four horizontal directions for cMidY.

> $$c' = \mathrm{lerp}(c, L(c) \cdot S / L(S), 0.6)$$

with c the authored colour, S the cube's colour in that direction and L() the luminance. The game's own blend by the normal then works on the new colours.

Fill light

The daytime states give shade a lot of light: on a sunny day in Zurdania the light from above is × 3.9 against the sun's × 5.15. Compared with neural-rendered shots of the same scenes, what those darken is exactly this fill. The card scales the three ambient multipliers of every day, dusk and dawn state in initial.pak (199 states; night, garage and menu states stay) by one factor f. Because the fill is about half of all light even on sunlit ground, less of it darkens the whole picture and the game's eye adaptation would lift everything back. So each state's exposure is scaled too:

> $$\mathrm{ambient} \times f, \quad \mathrm{HdrExposureScale} \times f^{0.45} \quad (f = 0.7 \to 0.852;\ 0.85 \to 0.929)$$

That holds sunlit parts about where they were and leaves shade darker. The edit is made in initial.pak by the engine (see the engine chapter).

Object reflections: the reflection pass (SSSR)

Card: Object reflections, with SnowRunner Shadows. Paint, glass and chrome mirror the scene around them. The name SSSR comes from AMD's FidelityFX Stochastic Screen Space Reflections, whose ray walk the pass adapts.

What the game does without it

A glossy material in SnowRunner reflects one thing: a cubemap, a picture of the sky in six square faces, the same for the whole map. The material looks it up in the direction of the mirrored view ray and blurs it by roughness through the cubemap's mip chain: mip level = √roughness × 8 + 0.5. A truck's hood therefore shows a painted sky even when a wall stands next to it. Screen-space reflections add what the camera can see.

The principle

For each pixel we know its position P, the direction to the eye V and the surface normal N. A perfect mirror sends the view ray on in the direction

> $$R = 2\,(N \cdot V)\,N - V$$

If that ray hits something the camera also sees, the colour of the hit is the reflection. The scene the camera sees is known as two pictures: the lit colour and the depth of every pixel. So the search is a walk along R across the screen, comparing the ray's depth with the depth stored at each pixel it passes, until the ray goes behind a surface.

Where normal and roughness come from

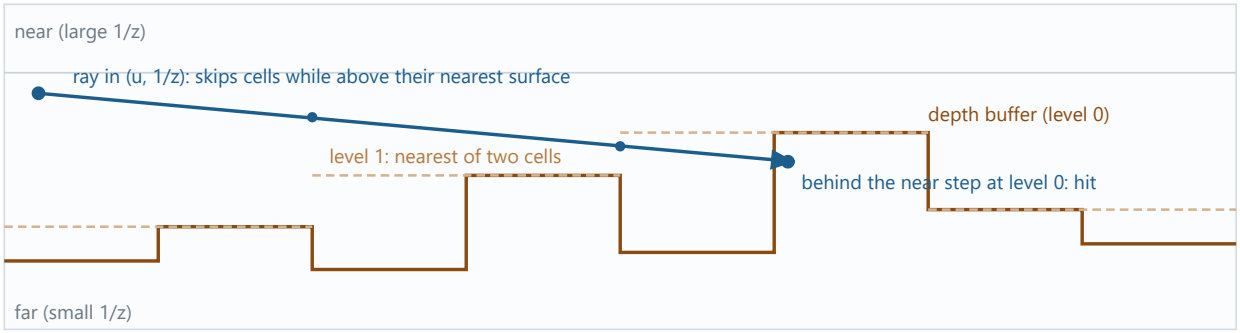
The game draws its world in one pass where each material computes its final colour; it keeps no separate buffer of normals. Next Gen's module sssr splices a small writer into all 3229 vehicle and prop material shaders, at their cubemap sample: it writes the shaded normal (world space, × 0.5 + 0.5) and the perceptual roughness r to an extra render target, RT7 (RGBA8), which the DLL binds beside the lit pass and clears to "up, fully rough" after it. A pixel with r ≥ 0.999 traces no ray.

Straight lines on screen, and why 1/z

The ray from O = P + N (0.002 z + 0.01) (lifted off the surface by 1 cm plus 0.2 % of its distance z, so it does not hit itself) is projected to screen coordinates at both ends. A straight line in the world stays a straight line on screen, but its depth z does not change linearly along it; the reciprocal 1/z does. So the pass keeps 1/z everywhere: the ray is o + t d in (u, v, 1/z), and the depth pyramid stores 1/z too. A larger 1/z means nearer.

The depth pyramid (Hi-Z)

Level 0 holds 1/z of every pixel (0 for the sky, z ≥ 3000 m, so no ray meets it). Each coarser level halves the size and keeps, for every cell, the *largest* 1/z of the four (or, at an odd edge, six or nine) cells below: the nearest surface anywhere in the cell. A ray that is nearer than that value cannot hit anything in the whole cell.



The walk through the pyramid, in one screen axis. Over cells whose nearest surface lies behind the ray, the ray takes the coarse level's long steps; where it would cross a surface it drops a level, down to single pixels.

The walk

The ray starts at level 0 and moves to the far edge of the pixel it starts in. Then, at most 80 times (SSRSteps):

1. Read the cell's nearest surface s at the current level.
2. Find where the ray leaves the cell: the nearest of its crossings with the cell's two edge lines and, while the ray moves away from the camera, with the plane $1/z = s$. That is t_{min}.
3. If the ray is in front of the cell's nearest surface (s < its own 1/z), move it to t_{min}. If it left through an edge rather than through the surface plane, go up one level: the next cells are bigger. Otherwise go down one level.
4. Stop when the level drops below 0: the ray stands behind a surface at single-pixel size. Or when it leaves the screen.

A walk over open ground passes many pixels in a few steps; near geometry it slows to pixel steps. This is the method of Uludag's Hi-Z tracing, in the form AMD wrote for FidelityFX SSSR, whose functions FFX_SSSR_InitialAdvanceRay, FFX_SSSR_AdvanceRay and FFX_SSSR_HierarchicalRaymarch the pass adapts (MIT licence, notice in the DLL).

Is it a real hit?

A point behind the depth buffer is not always on the surface: the buffer only stores the front of things. The pass checks, at the pixel where the walk ended:

Test	Rule
on screen, and moved	the hit lies inside the screen and more than one pixel from where the ray started
not sky	stored depth z_s below 3000 m
a front face	the hit surface's normal N_s faces the ray: $N_s \cdot R \le 0$
thickness	how far behind the surface the ray stands, $b = z_{ray} - z_s$, against the assumed thickness of things $T = 0.3\ \text{m} \times (1 + 0.02\, z_s)$

The thickness gives the hit a confidence c between 0 and 1:

$$c = (1 - \text{smoothstep}(0, T, b))^2$$

A ray more than T behind the front went under the object: under a cab, past a load on a bed. Leaving those pixels empty would let the material fall back on the painted sky, which looks like bright patches beside real reflections. So within 2.5 m (plus 5 % of the depth, SSRUnder) behind the surface, the ray takes that object's colour, darkened towards half (SSRUnderShade 0.5) over half a metre past T. This stands for the underside the camera cannot see. It applies in full to surfaces up to 25 m away and fades out by 50 m (SSRUnderNear, SSRUnderFar). It is not used behind a "speck", a small bright thing with nothing around it, such as a firefly or a distant lamp: on at least three of four sides, 2 % of the screen height away, the depth differs and the pixel is three times brighter than its brightest neighbour.

The confidence then fades at the screen's border (over 5 % of its height), with roughness, and for rays turning back towards the camera, whose targets no picture holds:

> $$c \times= \text{saturate}((r_{max} - r) \,/\, (0.2\, r_{max})) \times \text{saturate}(2\,(R \cdot -V + 1)),\;\; r_{max} = 0.85$$

Roughness: the cone

A rough surface reflects a blurred picture. By default every ray is the mirror direction, and roughness only blurs the colour taken at the hit, from the feed's mip chain. The blur grows with the distance the ray travelled on screen, d (pixels), and with $\alpha = r^2$:

> $$\text{mip} = \log_2(1 + d \cdot \alpha \cdot 0.5) \quad (\text{SSRCone} = 0.5)$$

Optionally (SSRLobe = 1) rays follow the GGX distribution itself: each frame draws a different microfacet normal H from the distribution of normals visible from V (Heitz 2018), and reflects about H instead of N. Two uniform numbers u_1, u_2 (interleaved gradient noise, Jimenez, shifted per frame) become a point on a disc that is stretched by α and lifted onto the hemisphere. Only the lobe's core is used (u_1 scaled by 0.85) to keep rare far-off rays from making speckles. This is the "stochastic" in SSSR; it is off by default because the mirror direction with the cone blur is steadier in SnowRunner's motion.

Fewer rays: one per 2 × 2 block

With SSRHalf = 1 (the default) the trace runs on a grid of half the width and height. Each cell traces one pixel of its 2 × 2 block, and the chosen pixel rotates over four frames in the order (0,0), (1,1), (1,0), (0,1), so every pixel gets its own ray every fourth frame. That quarters the trace's cost.

The resolve: from rays to a picture

The resolve runs at full size. Each reflecting pixel gathers the results of the 3 × 3 trace cells around it, since a neighbour's ray stands for a parallel ray from a nearby point. Each one is weighted by distance, depth and normal, so nothing crosses an edge:

> $$w = \exp(-|\Delta p|^2 \,/\, \sigma^2) \cdot \text{saturate}(1 - |\Delta z| \,/\, (0.03\, z + 0.02)) \cdot (N_s \cdot N)^{16},\;\; \sigma^2 = 2\, s^2\, (1 + 8\, r^2)$$

where s is the grid step (2 with SSRHalf). From the weighted samples come the mean μ and the spread σ (standard deviation) per channel.

Then the history. The pixel is found where it was last frame: by the camera's reprojection, or, on a moving truck, by the object motion decoded from the game's own velocity buffer. Last frame's result there is used only if last frame's depth agrees: within 2 % + 5 cm for still things, 10 % + 30 cm along a moving object. The history is then pulled toward this frame's mean until it lies within the samples' spread (variance

clipping, Salvi 2016). All channels move by the same factor, so the result stays a blend of colours that are actually there:

> $e = \max(1.5\,\sigma,\ 0.01 + 0.02\,|\mu|),\quad m = \max \text{ over channels } |h - \mu| \,/\, e,\quad \text{if } m > 1\text{: } h = \mu + (h - \mu) \,/\, m$

> $\text{result} = \text{lerp}(\mu, h, k),\quad k = \text{lerp}(0.6, 0.9, \text{saturate}(r \,/\, 0.3))$

Mirrors keep little history (k = 0.6); rough paint, whose rays are noisier, keeps more (up to SSRTemporal = 0.9). Where no ray landed near a pixel this frame, its history fades by 0.9 per frame. The output, t123, holds radiance × confidence and the confidence (RGBA16F).

Back into the material

The pass ran during the previous frame, so the material shader reprojects its pixel into that frame with last frame's view and projection, and checks last frame's depth there (3 % + 5 cm; 15 % for pixels a moving object passed). Then it composites, with a the confidence:

> $\text{colour} = \text{reflection} + \text{cubemap} \times (1 - a)$

faded by a gloss gate: in full up to cubemap mip 2 (very glossy), the cubemap alone from mip 7 (roughness about 0.67). Reflected light brighter than four times the cubemap's light is compressed, so a lamp in the reflection does not burn out. Glass is a special case: the pass sees the cab behind the window, not the window. Where the depth the pass saw lies clearly behind the fragment (1 % + 2 cm + 1.5 pixels of the fragment's own depth slope), the material marches its own short ray instead (the per-material method of the older Object reflections build).

Cost, limits, artifacts

- Cost: the resolve takes 0.21 ms at 3840 × 2160 (measured; it is bound by its full-size reads and writes). The trace is a quarter-size pass with up to 80 steps per ray. Per-pass timings can be logged with SSRTiming = 1.
- Only what is on screen can be reflected. Things outside the picture, behind the camera, or hidden behind other things are not in the depth buffer; the reflection fades to the cubemap there, most visibly at the screen's edges.
- The thickness is a guess: thin poles may be missed; a ray may pass under a thin object and catch the ground behind it.
- One frame of delay: fast camera turns show the reflection catching up for a frame, and newly revealed pixels have no history.
- F6 in the game switches the pass off and on; with it off, the material marches its own ray (or keeps the cubemap).

Water: reflections, Fresnel, depth colour, crest glow

Cards: Water reflections (row Sun glitter in shade), River depth colour, Sun glow through waves. The game draws water with about 200 shaders: 20 for rivers and mud, 176 for lakes and seas ("domain" water), 88 of those with a planar reflection image. Next Gen changes what they reflect, how reflection and body colour mix, and how deep water is coloured.

Screen-space reflections on rivers and plain lakes

The river and mud shaders, and the lakes without a planar image, reflect only the sky cubemap. The patch replaces their one cubemap sample with a ray march (ssr.hlsl, after McGuire and Mara, "Efficient GPU Screen-Space Ray Tracing", JCGT 2014, simplified for water, whose rays leave the surface upwards). Unlike the object pass it marches in view space with steps that grow geometrically, which suits long reflections over flat water:

> $\text{step}_k = 0.05\ \text{m} \times 1.25^k,\ \ k = 0 \ldots 35\ \ \ (\text{the last step is } 0.05 \times 1.25^{36} \approx 150\ \text{m})$

The first step is small so a wheel standing in a puddle is found. At each step the ray's point is projected and its depth compared with the scene's linear depth (g_txZ). When it passes behind a surface, 5 halvings between the last point in front and the first behind find the crossing. A hit counts when the ray is at most 0.25 m behind the surface (deeper, it went under the object); the bias is 0.02 m plus a share of the distance. The colour comes from the opaque scene the game already has (g_txBBOpaque). Hits brighter than 3 times the reflected sky are compressed, so lamps do not burn out; the reflection fades over the outer 8 % of the screen. Where the ray leaves the screen or finds nothing, the sky sample stays exactly as before.

The lakes' planar image

88 lake, sea and puddle shaders reflect an image the game renders for one water plane, at half the frame's size, with its own screen-space pass (SpinTires/WaterCompute, "Reflections" and "ReflectionsWithTAA"). For every water texel that pass walks up the screen one texel a step, finds the first thing standing out of the water, and takes the colour twice as far up. Its hit rule has a flaw: it asks that what it finds rises at least minRise = 0.01 + 0.04 s metres *per texel* (s grows with the depth). An upright thing 20 m away rises 3.0 cm per texel at 1080 rows but only 1.5 cm at 2160 rows, so at 3840 × 2160 a hull, a tyre or a pier standing in the water is found only beyond about 75 m, and the lake shows sky where a truck should be. The patch (patch_refl_hits.js) makes the rule independent of resolution:

> $\text{minRise} = 0.5 \times (\text{the height one texel covers at the sampled depth} - \text{tol})$

On top, the same ray march as the rivers runs at the planar sample (ssr_planar.hlsl). Where it hits, it replaces the planar image (which leaves out the trucks); where it finds nothing the planar sample stays. A ray that ends well behind a surface is not handed to the planar image: under an object it takes that object's colour, darkened, as the object pass does.

How reflection and water colour mix

Light at a water surface splits: a share F is reflected, the rest enters the water. F follows Schlick's approximation of Fresnel's equations, with F_0 = 0.02 for water:

> $F = F_0 + (1 - F_0)(1 - \cos\theta)^5$

The game's river shaders end with out = reflection · F + colour, adding the reflection on top of the full body colour. That works while the reflection is bright sky; with real reflections of darker banks the water turns milky. The patch (blend.hlsl) makes the colour give way:

$$\text{out} = \text{colour} \cdot (1 - \text{F}) + \text{reflection} \cdot \text{F}$$

except for the sun's highlight, which the larger variants add into the body colour and which is reflected light already: its share, highlight · (1 − T), stays as it is.

River depth colour: coloured absorption

The game fades the riverbed behind the water into the water's body colour with one grey factor, T = exp(−k · path), so deep water goes grey at the same rate in every channel. Real water absorbs some wavelengths faster (Beer-Lambert): clear water loses red first and deepens to teal, muddy water loses blue first and goes brown. The patch (absorb.hlsl) gives each channel its own exponent, shaped by the water's own colour c (its lit body colour with the sky's tint divided out):

$$\text{rel}_c = c \,/\, \max(c), \;\; a_c = 1 - \ln(\max(\text{rel}_c, 0.02)), \;\; w_c = a_c \,/\, L(a)$$

$$\tau = -\ln T, \;\; \tau' = \tau^2 \,/\, (\tau + 0.7), \;\; T_c = \exp(-w_c \, \tau')$$

The brightest channel gets a = 1 and darker ones more; the weights average 1 by luminance L, so the overall depth the artists set stays and only its hue changes. A grey water colour gives the stock fade. The second line clears the shallows: thin water (small τ) lets the bed show much further, deep water fades as before.

Sun glow through waves

Looking towards a low sun across a river, the thin tops of waves glow: light enters the back of a wave and leaves through its crest. The term is the "fast translucency" of Barre-Brisebois and Bouchard (GDC 2011): the direction to the sun, bent by the wave's normal, against the view direction:

$$\text{glow} = 0.15 \cdot C_{\text{sun}} \cdot S \cdot \text{hue} \cdot \text{crest} \cdot \text{saturate}(V \cdot -(L + 0.5\,N))^6$$

with S the sun's shadow-map visibility at the pixel (looked up as the game's materials do; the 12 river shaders that read the shadow map get it), hue the water's own colour, and crest a weight that rises as a facet tilts away from flat (a tilted facet is a thin crest; factor 4). It is added outside the reflection, as light that came through the water.

Sun glitter in shade (row, off by default)

The game's highlight term on water ignores part of the sun's shadow: water in shade still glitters a little. The row multiplies the highlight's power term, in the 64 lake and 8 river shaders drawn with the shadow map, by

$$g = \text{saturate}(1 - (1 - S) \,/\, \max(1 - f, 0.001))$$

with S the shader's own shadow test and f the game's shadow floor (g_fShadowAtten): where the test says the sun is hidden, the highlight goes out. Where it was tried, the game's test said the water was lit, so nothing changed; it stays off.

Limits

- Screen space: what is off screen is not mirrored; the reflection fades at the screen's edges.
- The march samples in steps: very thin things (a rope) can be stepped over at a distance.
- The glow needs the shadow map; 8 of the 20 river shaders do not read it and get no glow.

Wet ground reflections and the glare cap

Cards: Wet ground reflections, Headlight glare cap.

Reflections in mud and puddles

The terrain lights its reflections from the sky cubemap alone (scaled by the local ambient light), so a puddle next to a truck mirrors the sky and nothing else. Terrain is drawn before the game has an image of the scene, so the patch (puddle_ssr, in 1208 wet-ground shaders) reads the previous frame's, which the DLL copies at the AO pass and binds at t120 (colour) and t121 (depth). The ray is marched in world space and projected with the previous frame's camera, so the depth and the colour it meets belong together; for a moving truck the reflection lags one frame. Only glossy pixels march: the cube's mip level, √roughness × 8 + 0.5, must be below a limit. Rough mud takes its colour from the feed's mip chain, blurred like the cubemap would be. Without the DLL (or with F9) every pixel keeps the cubemap sample.

The headlight glare cap

The terrain lights every surface with a GGX highlight per headlight. Its distribution term D is capped at 32 by the game: min(D, 32). On the smooth, rippled surface of a road puddle that cap is reached along a long stretch in front of the truck, and the lamps draw a long bright band across the water. The patch (patch_glare.js, about 1216 shaders) lowers the cap for the headlights only:

> $\min(D, 32) \rightarrow \min(D, 6)$ (the headlight terms; the sun's and the baked lights' keep 32)

A cap belongs to a headlight when a read of that lamp's colour (g_cHeadLightColor[k]) lies between it and the previous cap. The number is rewritten in place. Rough surfaces, whose highlight never gets near the cap, look as they did.

Headlights in reflections

Card: Headlights in reflections. At night, paint that finds nothing to reflect shows the light of the headlights.

Where a reflected ray from the object pass finds nothing on screen, the material falls back on the sky cubemap: at night, a dark blue. But the ray passes through the headlights' beams, which light the air (the game draws that as its own beam). This build of the object reader adds, along the reflected ray, the beam's light in the air: the cone, reach and colour come from the game's own constants (CB_GLOBAL_SCENE), so a hood at night shows the lamps' warm light where its rays cross the beam. With the lamps off, or the pass unbound, the result equals the plain reader's at no cost. With two lamps on it adds 0.15 ms for a whole 3840 × 2160 screen of glossy pixels (measured on an RTX 4080); a real frame has far fewer.

Volumetric fog and sun shafts

Card: Volumetric fog (off by default), with its row Sun shafts (off by default, experimental).

How the game draws fog

The fog pass (shader 0x871EF8CC) marches along each view ray: 128 steps of 0.5 m (0.5 + 0.1 × the lighting mode) from a start depth. At each step the density is a fog mask times a height falloff; the colour is a shadow colour plus the light of lamps and headlights. Two things show. Every pixel samples the same depth slices, which at grazing angles shows as layers or bands. And the light does not depend on the sun's direction: fog looks the same towards the sun and away from it.

Jitter

Every pixel starts its march up to one step later, by an interleaved gradient noise value n ∈ [0, 1) on the fog target's coordinates:

> $t_i = t_{start} + (i + n) \cdot \Delta t$

Neighbouring pixels then sample different slices; the bands become fine noise that the fog buffer's filtering smooths out.

Forward scattering

Fog droplets scatter light mostly forward, so fog between the eye and the sun glows. The standard model is the Henyey-Greenstein phase function, with θ the angle between the view ray and the light's direction and g the anisotropy:

> $p_{HG}(\theta) = (1 - g^2) \,/\, (4\pi\,(1 + g^2 - 2g\cos\theta)^{3/2})$

The patch multiplies the in-scattered light (not the density, so the fog's opacity stays) by a blend of isotropic and Henyey-Greenstein scattering, relative to isotropic (1/4π):

> $P = (1 - K) + K \cdot p_{HG}(\theta) \,/\, (1/4\pi), \;\; g = 0.3, K = 0.4 \;\Rightarrow\; P = 1.66 \text{ towards the sun, } 0.77 \text{ away}$

Sun shafts (row)

The stock march never asks where the sun is blocked, so there are no shafts between trees. The row splices a helper into each step of the march (fog_sun.hlsl) that adds the sun's in-scattered light:

> $L_{sun} = 0.12 \cdot C_{sun} \cdot S(P) \cdot \text{cloud}(P) \cdot p_{HG}(\theta; g = 0.45) \,/\, (1/4\pi)$

added into the march's sum with the step's density, the way the stock colour goes in. S(P) is the sun shadow map at the step's position, looked up as the game's own materials do: the view depth picks the cascade (the split distances), the position is projected with that cascade's matrix into the atlas, and one bilinear comparison sample is taken. cloud(P) is the cloud shadow map (1 − its red channel at the position's map coordinates).

Limit: the fog is drawn without knowing what stands in front of each step, so with a low sun the shafts also show over the truck and other things close to the camera. That is why the row is experimental and off.

Tonemap, bloom and photo grade

Cards: Tonemap, Bloom soft cut, Photo grade. Three steps at the end of the frame, in the game's post-process shaders (common_pc_sm50_hdr.sdc) and its colour tables.

Tonemapping

The scene is rendered in linear light, values from 0 to far above 1. A tonemap curve turns them into the 0 to 1 the screen shows. The game uses a filmic operator of Hable's form, with parameters per daytime state (TonemapA to F and a white point W in the daytime files):

$$f(x) = (x\,(A\,x + C\,B) + D\,E)\,/\,(x\,(A\,x + B) + D\,F) - E\,/\,F, \quad \text{out} = f(x)\,/\,f(W)$$

Next Gen's tonemap is a reconstruction of the game's shader with a different curve (CURVE 3): the same operator with its toe term off and no white point, three constants fitted to a target response:

$$y = (x^2 + 0.20\,x)\,/\,(x^2 + 0.19\,x + 0.32), \quad x = 0.18 \text{ at mid grey}$$

The exposure is matched to the game's own curve, so the scene value each daytime sends to mid grey still lands there: each state keeps its authored brightness and only contrast, toe and shoulder change. Measured per photographic stop around grey (in 8-bit sRGB output):

	game's curve	Next Gen
contrast (slope) at mid grey	0.91	1.22
output three stops under grey	43	34 (deeper shade, still with detail)
output two stops over grey	191	216 (brighter highlights, rolling off without a clip)

The bloom texture's weight in the final composite is set to 60 % (BLOOM_SCALE 0.6).

Bloom soft cut

Bloom adds a glow around bright things. The game's knee pass (0x52879E18) weighs each sample by how far it exceeds a threshold:

$$k = \max(L \cdot \text{exposure} - \text{threshold}, 0), \quad w = 10 \cdot (0.1\,k)^{\gamma}, \quad \gamma = \text{BloomGamma} = 0.6$$

An exponent below 1 lifts faint values: snow in daylight (k ≈ 1.4) gets w ≈ 3.1, which is the grey haze over bright surfaces. The patch keeps that code and multiplies each weight by a soft knee:

$$w' = w \cdot k^2\,/\,(k^2 + 3^2) \quad (\text{snow} \times 0.18, \text{sky} \times 0.4, \text{sun and headlights} \approx \times 1)$$

Real highlights, whose k is far above the knee, still bloom.

Photo grade

The game finishes the picture with a colour table per daytime state: a 16 × 16 × 16 LUT (in boot.pak), sampled with the display-encoded colour after the tonemap. Four tables carry the 161 day, dusk and dawn states (day_us_01 for 64 of them, identity for 44, day_us_12 for 34, day_us_18 for 19). The grade was measured on four pairs of screenshots, the game's own against neural-rendered shots of the same scenes: those take 13 to 21 % of the colour out, most from the greens (× 0.70) and the autumn oranges (× 0.75),

less from sky blue (× 0.86 to 0.9), and set whites a little lower. The grade is applied to each table's output texel:

The grade also takes out the game's lens dirt. The final post pass adds a texture of soft discs (env_camera_bokeh, 1024 × 512) over the picture, scaled by the light where it falls, so over bright sky and clouds it shows as blurry circles that follow the camera. The texture's texels are written as zeros (black, no alpha), so the pass adds nothing; Restore and the grade switched off put it back.

> $$\mathrm{LUT}'(c) = \mathrm{grade}(\mathrm{LUT}(c))$$

so the game's own look stays underneath, and greys stay grey. The row Strength scales the grade (100 % is the measured one). The table file is 82 header bytes, then 4096 texels as B, G, R, A with red fastest, then 6 bytes; the entries are stored, so boot.pak is rewritten with the same sizes.

Reflection cubes from photographs

Card: Reflection cubes. The painted sky cubes that paint and chrome mirror and the ambient light reads, made anew from photographs for every region and hour.

What a reflection cube is in SnowRunner

Each daytime state names one cube (EnvMap="env/day_al_01__d_ggxrefl.tga"; in boot.pak a .pct texture). It has six square faces of 256 × 256 texels in Direct3D's order +X, −X, +Y, −Y, +Z, −Z, compressed as BC7, with 8 mip levels down to 2 × 2, each a plain downsample of the one before. Materials sample it at mip √roughness × 8 + 0.5, so the mips stand for rougher and rougher reflections. Sky ambient and Bounce light read its last level as the sky's and the ground's colour. The game shares a few painted cubes between all regions. Next Gen makes 25 of the 26 the states name from 21 photographic panoramas (CC0, Poly Haven; the garage keeps the game's).

From a panorama to six faces

A panorama in equirectangular form maps every direction to a point of a 2:1 picture. For a direction d = (x, y, z), y up:

$$\varphi = \operatorname{atan2}(x, z),\quad \theta = \operatorname{asin}(y),\quad u = \tfrac{1}{2} + \varphi \,/\, 2\pi,\quad v = \tfrac{1}{2} - \theta \,/\, \pi$$

Each face texel is a direction (the texel's centre on the face of a cube around the origin, normalised); make_cube.js looks the panorama up at 3 × 3 points inside the texel and averages them, so the faces are not aliased. The source is Radiance HDR (RGBE: a shared exponent per texel), linear light.

Exposure, colour, and the game's brightness

A photograph has its own exposure. The cube is scaled so that its upper half (the sky) lands at the brightness of the stock cube's sky, an sRGB luminance (SKY_MEAN). Options set the sky straight overhead and the ground apart from the sky, and white-balance a dusk, dawn or night to the hue of the game's cube for that hour. Then the texels are encoded sRGB, as the game's are, and box-filtered down the mips: each texel of mip k + 1 is the mean of four of mip k.

BC7 encoding

BC7 stores each 4 × 4 block in 16 bytes. The tool uses mode 6: one subset, two RGBA end points of 7 bits per channel plus a shared low bit each, and a 4-bit index per texel that picks one of 16 points on the line between them. For a block of texels c_i, the indices are chosen by projecting each texel on the line, then the end points A and B are refined by least squares for those indices:

$$\text{minimise}\ \Sigma_i \left| c_i - ((1 - w_i)\, A + w_i\, B) \right|^2,\quad w_i = \text{index}_i \,/\, 15$$

which is a 2 × 2 linear system per channel. The file's 138 header bytes and its 6-byte tail are copied from the stock cube, so the game sees exactly the layout it ships with.

The winter cube rule

The game names cubes per state by family, and two of its choices stop holding once the snow families' cubes are real snow photographs. Kola and Yukon give their clear days 1 and 2 the Michigan summer cube and only day 3 the snow cube; with the rule, a clear day of a region with snow that names a summer cube

takes the snow cube its region's other clear days use. And a few states of regions without snow name a snow cube (a weather variant of Wisconsin, North Carolina's dusk); they take the cube their region's other states of the same kind use. The engine applies this to initial.pak whenever boot.pak holds the new cubes.

Night sky, brighter stars, skies

Cards: Night sky, Brighter stars, Skies.

Night sky: a real star map

The game's stars are one texture (4096 × 1024) wrapped around the sky, with stars a texel wide. Next Gen's is NASA's Deep Star Maps 2020 at 8192 × 2048: point stars at their catalogued brightness and the Milky Way's glow. The two photo night skies of Scandinavia, Kola and Quebec (one picture each, an aurora photograph) get the same star points painted in.

The glow is faint: at the galaxy's edge its light is 1 to 3 in the alpha's 255 steps, each step half again or more, which the layer's multiplier (up to × 5) showed as flat islands with hard edges along the band (seen in the game on Washington's night sky). Since the shader weights the colour by the alpha, the faint glow is stored as both: the alpha rounded up and the grey carrying the rest, so its light steps by alpha / 255 of a step. The glow under that is found again smooth from the texture itself (the smallest alpha in 5 × 5, which leaves the stars out, then a 17 × 17 box where it is faint and 5 × 5 where it is bright); the stars and the bright glow are as they were.

Brighter stars: why the level must rise

A sky layer's brightness is "g(r; g; b; a) × N": N scales the colour, the alpha is a / 255, and the sky shader outputs the colour weighted by the alpha. The night states draw the star layer at × 0.07 to × 0.5, levels set for the stock texture's wide stars. NASA's point stars put about a quarter of the light into each star's texel, so at the stock levels they come out at 1 to 2 nits (the brightest at 6 to 9) on a sky of 0.15 to 0.4 nits; the texture's alpha is already at its 8-bit ceiling for the bright ones. So the card multiplies N in the night overrides of the star layer by one factor (dusk and dawn keep their level where a region shows stars then, as they are bright skies).

The photo skies cannot take that factor: one picture holds the photograph and the stars, so N would brighten the photo too. Their build carries the stars' share in the alpha instead: 85 where a texel is all photo, up to 255 on star points. The layer's own alpha is raised three times (50 → 150): the photo stays as it was (85 × 150 = 255 × 50), the star points come out three times as bright.

Skies: domes and cloud bands from photographs

A SnowRunner sky is layers on a sphere: a sky dome (sphere_sky) and up to three cloud bands (clouds_rear, clouds_mid, clouds_near) that turn around the map, each tinted per daytime state. The card (on by default since 1.1.0) replaces 49 of those textures, every region's, with ones made from 25 photographs of real skies: domes by make_cube.js at the stock texture's size and mean colour with the photograph's sun taken out, and cloud bands. Two night layers that are black but for a patch of the Milky Way get the real one: a strip around the galactic centre sampled from NASA's Deep Star Maps 2020 through the galactic to equatorial rotation, laid along the stock patch's long axis (its moments give the centre, tilt and spread), faded by an ellipse of that spread, and its light mapped onto the stock patch's own distribution of brightness, quantile for quantile. The cloud bands are cut from panoramas by make_clouds.js: a band is a texture 8 (or 4) times as wide as high whose alpha is the clouds' shape and whose colour is their shading in grey. What is cloud and what is sky is told by colour, since a clear sky is blue and a cloud is not:

> $b = (B - R) / (B + R)$, $b_{sky} = 85\text{th percentile of b in the row}$, $\text{cloudiness} = b_{sky} - b$

The alpha is a soft step over the cloudiness, set so that the band's mean alpha matches the stock band's coverage; the band fades out at its upper and lower edge as the game's do; the colour is the texel's luminance over the brightest cloud's, so a state's tint works on it as on the game's. The alpha is softened over its neighbours, so a cloud's soft edge reaches out over texels that are sky in the photograph; their grey is the sky's, darker than the cloud, and showed at night as a dark rim round every cloud (seen in the game, 2026-10-23). Averaging the grey over the same neighbours, each weighted by its alpha squared, did not take it away: a thin edge is part sky in the photograph and came out darker than its cloud (219 inside, 186 at the edge, 173 just outside in a near band), and filtering between texels and every smaller mip mixes that in. Now every texel carries the grey of the clouds near it: each grey weighted by its alpha to the fourth (a solid cloud counts, a thin edge hardly), summed down a pyramid of 2 × 2 cells and pulled back up, so a cloud fades out by its alpha alone. The smaller mips average the colour by alpha too, so a clear texel's colour does not count where cloud is near. On the top level the clear texels beside a cloud were 5 to 19 levels of 255 below the soft edge; now they are at most 8 below it, and in some bands above it.

The zenith: every column of a dome meets at the top, and the game closes its dome above about 81 degrees with a fan of triangles, so whatever a photograph has up there is squeezed into wedges around a point with a ring around it (seen in the game looking straight up, and as streaks running to the zenith on the overcast skies). From 66 degrees up each row of a dome, and of a cloud band that reaches that high, is blended towards its own mean colour and alpha, smoothly, wholly at 86 degrees: the sky straight overhead is plain, as in the game's own textures. Yukon's night dome (sky_01a_us_04_night) keeps its first build for now: rebuilt with the options on record it does not match the shipped texture, so it is not rebuilt from a guess. The near cloud band most maps use (cloud_03_us_01, cut from a broken cloud field) is cut with a wider soft edge (0.45), its alpha softened over 3 texels and patches under 400 texels left out: at night its small hard-edged fragments showed as specks against the stars. Around the photograph's sun a band holds only the fill, with no cloud of its own; cut clear there it left a round hole with a hard edge in the band. That stretch now takes the band's clouds from the far side, half way round at the same height, faded in over the fill's edge.

The sun: the game draws its own, so the photograph's comes out, with its glare. The glare is measured as a gain over the sky on the far side, degree by degree out from the sun, and divided out. Each degree's gain comes from the ring of texels at that angle, ranked by brightness, from the ones a quarter of the way up: clouds near the sun are lit by it and stand above the sky, and the median of a ring half covered by them took the sky beside them too far down, a dark ring around the spot. The gains are eased from one degree to the next, since a step showed as a coloured ring. The disc itself, 2 to 6 degrees where the glare is over 30 times the sky, holds nothing usable and is filled over a 4 degree edge. The first fill carried each direction's colour straight in from the ring outside it, and a disc beside a cloud met at its centre as a pinwheel of cloud and sky with a ring around it (seen in the game on a clear day, 2026-10-23, where it looked like the zenith but stood at the sun's 48 degrees). Now it is a membrane over that ring: at the rim each direction keeps the colour that stands there, and towards the middle the differences fade, finer ones faster, to the ring's mean. On it goes the detail of a patch as big at the same elevation 60 to 300 degrees round the sky, as its brightness over that patch's own membrane (between half and twice), so a cloud deck carries on through the disc; of those patches, the one whose ring varies most like the disc's. Under a cloud sheet the glow is the sheet's own and stays, and the fill rises with it towards the middle, level from the disc's edge in: a level fill under the glow showed as a dark spot.

Where the fill still read as a patch in the game (a soft blurred disc where the sun had been, seen on 2026-10-23), and at the zenith, the spot is now repainted by an image model, Google's Gemini. engine\tools\sky_edit.js cuts flat views of a built dome: one centred on the sun's old place, 50 degrees wide with a circle of 10 degrees, and one straight up, 110 degrees wide with a circle of 35. Each goes to the model with the circle drawn in red and a prompt to repaint only inside it and take the circle out. Its apply puts the answers back. Each picture's colour is matched to the dome's own just outside the circle (the

model shifts a whole picture a little: Zurdania's three answers were off by 1 to 2 levels of 255 there), taken wholly up to half a degree past the circle and faded out by 4, the zenith's view first, so the sun's edit has the last word where they meet. Then the cap: from 64 degrees up blended towards one colour, wholly at 80, the median of the sky between 76 and 80 degrees (their mean, with a cloud reaching the ring, left a lighter disc, and a blend over 7 degrees still showed its edge). Every level is encoded again from the edited one, and a before and after of each view is written beside it. The first two: Zurdania's clear day (cloud_04_ru_02_day_01, the sun at 48 degrees and the zenith) and its overcast (cloud_04_ru_02_day_02, the zenith), in the game since 2026-10-23. A dome built again from its panorama loses its edits until apply runs on it again; the views, the answers and their records are kept in out\gemini.

Three overcast domes were made again after they read as plain in the game (2026-10-24), where the game's own are heavy, stormy skies: Kola's day__2a, which its dawn uses too (sky_02a_ru_03_day), had come out as a fair sky of small clouds with a round hole where the photograph's sun had been taken out; Kola's day__3a (sky_03a_ru_03_day) as thin streaks running to one point near the horizon; Zurdania's overcast (cloud_04_ru_02_day_02) as an even grey. They are now made from Kloppenheim 07, Overcast Soil and Mud Road. Overcast photographs are flat (all eight of Poly Haven's pure overcast skies are marked low contrast), so make_cube.js sky takes --contrast k, which spreads each texel's light around the sky half's geometric mean luminance, $L' = G\,(L / G)^{k}$, the colour kept, before the mean colour is matched: 1.3, 1.4 and 1.6 here. No sun is taken out of them: their glow is soft, as in the game's own overcast textures. Built by out\skies_build\overcast.ps1.

The aurora: the two layers that hold it (cloud_01_ru_03 and cloud_01_us_11, the second the first's mirror image to within about 6 levels of 255) had it blue and violet, green only along its lower edge and in a curl. A real aurora is mostly green, the light of oxygen at 557.7 nm, often with red above it where the oxygen is higher and thinner (630 nm). It was cut out as a square on black, 2080 texels wide, and Gemini repainted it in those colours with its shape and curtains kept. The answer goes back scaled to the square, its black floor (3 of 255, JPEG noise that would veil the sky) taken off, its colour strengthened 1.6 times around each texel's own grey and its total light set to the stock aurora's (a gain of 0.78). The second texture is the first mirrored, and both keep the stock alpha. The answer, the crop and the scripts are kept in out\gemini\aurora.

Each texture goes in with its .pct_header, which the game keeps beside every .pct: the .pct's header with its last record (type 0xff, the length of the data after it) turned into type 1 with the header's own length, as all 349 pairs in the stock boot.pak have it. Until 2026-10-23 the tools wrote the .pct's head as it stands, and the domes made smaller to the stock size (4096 × 1024) kept an older header of 8192 × 2048; now every pair agrees.

Smoke: glow, shape, softer edges

Cards: Smoke glow (row More kinds of smoke), Smoke shape and softer edges.

How the game lights a puff

Exhaust, dust, steam and smoke are sprites: flat textured squares that always face the camera. The game lights each with one sun value and one ambient value per vertex: either a baked top-down light map, or the sun's colour times a visibility factor. Nothing depends on where the sun is relative to the camera, and the sprite has no shape: a puff is a flat disc of light.

Glow: forward scattering

Smoke scatters sunlight mostly forward, so against a low sun it glows, often brighter than the sky behind it. The helper, spliced into the particle vertex shaders, multiplies the puff's sun light by

$$f = \min(1 + 0.15 \cdot \max(4\pi\, p_{HG}(\mu;\, g = 0.7) - 1,\, 0),\, 2)$$

with μ the cosine between the view ray and the sun's light direction and p_{HG} the Henyey-Greenstein phase function (see the fog chapter). $4\pi\, p_{HG}$ is 1 for isotropic scattering, so f = 1 from 90 degrees away from the sun on, and rises towards the sun (looking straight at it, $4\pi\, p_{HG} \approx 18.9$ and the cap of 2 applies). The factor rides on the sun term, so the pixel shader's shadow test, the light map's baked shade, the night fade and the fog still apply. Particles whose vertex shader gives them no sun light at all get the sun's colour times f − 1, the glow alone.

The particle vertex shaders are of a few families, and the patch has to find the sun term in each: a light map decoded from one register lane, the sun colour added, the sun colour times a visibility (plus the ambient). Of 205 vertex shaders, 162 take the glow by default. The row More kinds of smoke adds 8 whose light map decode sits in lanes x, z and w of its register (170 of 205). Of the other 35, 10 are normal-mapped puffs (lit in the pixel shader), 22 output no light, and 3 lack the scene constants.

Shape: a pseudo-normal from the alpha

The pixel-shader part (smoke_shade, in the 80 lit, blended puffs without a normal map of their own) gives each pixel the normal of a soft ball. Its direction on screen is the gradient of the puff's alpha (outward, where the alpha falls off); its tilt comes from the alpha itself: a thin rim faces sideways (sine of the tilt 0.9 at alpha 0), a dense core faces the camera. With f the flat sprite's normal (towards the camera) and L the direction to the sun, the lit colour is scaled by

$$k = (1 + 0.3\,(n \cdot L - f \cdot L) + 0.15\,(n_y - f_y)) \times (1 - 0.12 \cdot \mathrm{smoothstep}(0.5, 1, \alpha))$$

The side towards the sun gains what the far side loses, tops are lighter than bottoms (sky light from above), dense cores a little darker (light lost inside the puff). A puff seen face-on in its middle keeps its stock light. Only the lit part is scaled, the fog blend after it stays: out = c + (k − 1)(c − fog.a · fog.rgb).

White smoke is drawn thinner: at the game's opacity it comes out too white and too dense. Where the puff's own colour (its albedo, before light) has a luminance from 0.1 to 0.3, its opacity goes from the game's to 0.6 times it. Black exhaust keeps the game's opacity.

Softer edges

Where a sprite cuts through the ground it shows a hard line. The game fades sprites against the depth behind them; in the 110 shaders that do, the fade now runs over twice the distance, so puffs meet the ground softly.

Sharper smoke and particles

Card: Sharper smoke and particles.

The game's 40 particle textures (the sfx_* files: smoke, dust, sparks, spray) are replaced with versions at twice their width and height, upscaled by the author with NVIDIA's DLSS 5 Visual Enhancer and stored in the game's own texture format (.pct, with its .pct_header). Each replaces its entry in boot.pak, stored as the game stores it. They remain Saber Interactive's assets and ship only for use with the game. Cost: four times the texture memory for these 40 textures.

Weather, and debris and splashes

Cards: Weather (nine rows), Debris and splashes. Both edit the game's own XML in initial.pak: what each daytime state carries, and how each particle type behaves.

What the game's states carry

The game cycles each region through its day states and their weather variants (day__1, day__1a, on newer maps up to day__1e). A state can carry rain or snow, <WeatherParticles Type MinIntensity MaxIntensity IntensityChangeInterval>, whose intensity is drawn anew within the range every interval, and moving cloud shadows, <CloudShadowMap Map Scale Speed Direction>. In the stock game, 16 regions have rain or snow in their variants, only the snow regions have weather at dusk and night, and only 39 of 199 states have cloud shadows.

The rows

Row	What it writes
shadows	cloud shadows in every day state that has none, as Zurdania has them (scale 6, speed 2, direction (1; 0.5)): the light map for plain states, the medium one for variants, the dense one for heavy rain or snow
showers	rain and snow swell and ease: an intensity range (min, max) becomes (0.3 min, 1.2 max)
evening	regions with rain by day get a light drizzle at dusk and dawn (0 to 0.15) and at night (0 to 0.2)
horizon	the horizon cloud layer that 17 sky files hold commented out, put in, with overrides written for the file's own states
far	rain and snow drawn 1.5 times as far from the camera (32 m → 48 m)
fireflies	fireflies at night in the rainy regions: a new weather type with the game's own firefly sprite, drawn to 90 m at an intensity of 0.0005 to 0.001
pollen	pollen drifting in the clear days of the same regions, drawn to 64 m at 0.01 to 0.05
drift	clouds the game holds still turn slowly
land	the land drawn as far as the distant fog reaches: where a state hides terrain (TerrainHideDist) nearer than its fog ends, the distance goes up to the fog's end (on by default)

Fireflies and pollen are weather types of the game's later engine (Expeditions); here they are two new files in classes\weather. Every attribute written is one the game's own files use.

Debris and splashes

Particle types are small XML files (one <CombineParticles> each). The card applies one rule per family:

- Fragments (leaves, grass, twigs, dirt, stones thrown by the wheels): stock they vanish after 3 or 4 s. Now they live 30 to 40 s (LifeTime 30, Random 10) and collide with the ground, the truck and its wheels, with a 1 cm radius so a piece lies flat and a little bounce. Dandelion seeds float, so they only live longer (12 to 16 s).
- Splashes (mud, water, snow, dirt bursts): a splash throws D times its drops (Subparticles Density, D = 4 by default, between 12 D and 64 D), the drops lie twice as long, the splash lasts twice as long and leaves twice the mud and water marks on the truck.
- Puffs (exhaust, dust, steam) hang in the air half as long again (LifeTime × 1.5).
- Snow splashes last three times as long, and their drops fall at half gravity with 0.6 of the air resistance and live 15 s: light snow hangs in the air.
- Water spray: what a wheel throws spinning in water (water_splash_small, water_flow) and its drops (water_splash__subparticles, water_droplet__subparticles) fly further and are bigger. Stock the spray stops within half a metre: its air resistance is 3.0. The air resistance of the spray and its drops is now 0.4 of the game's (3.0 to 1.2, 1.0 to 0.4, 1.5 to 0.6), the spray's sprites are 1.5 times the size and live three times as long, the drops 1.5 times the scale. How hard the wheel throws stays the game's; with linear air resistance k a throw carries v / k, so the same throw goes about 2.5 times as far.

Cost: more particles alive at once, so more to draw; D is 4 by default.

Scenery detail and grass reach

Cards: Scenery detail (rows: the near switches, hide, shadows), Grass reach.

Levels of detail

Each mesh carries simpler versions of itself and the distances at which the game switches to them. Many meshes swap 10 to 15 m from the camera. With GTAO and sharper shadows, that swap shows as a flip in shading at a fixed distance, rock outcrops most of all. The meshes live in shared.pak (2 GB) as "[meshes]\models_<name>" and "[meshes]\plants_<name>": each a 32-bit size, an XML header, then binary data. The header holds the switches:

> <MeshLod Distances="10, 25, 45" ... />

A mesh whose first switch is nearer than 40 m gets its switches raised to at least 40, 70, 110 and 160 m; a value already further stays. The card's set is either "nature" (rocks, trees and bushes by name; stumps, twigs, crops, ruins, walls and signs left out) or every mesh with switches. Two extra rows take out the distance beyond which a mesh is no longer drawn (HidingDistance), and the tag that ends a small thing's shadow or keeps it from casting one (MeshShadow), for plants or for everything.

The XML must keep its byte length, so the binary data after it and its size prefix stay byte for byte: the whitespace between tags gives or takes the difference.

Re-compressing only what changed

shared.pak is a zip; many of its entries are deflate-compressed, and a mesh's XML sits at the start of its stream. A deflate stream is a series of blocks, each with its own Huffman tables. Re-compressing whole meshes would take long; the tool instead re-encodes only the blocks that hold the XML header and appends the rest of the original compressed bit stream as it is, shifted to its new bit position (a block need not start on a byte boundary). The result is inflated again and checked before the pak is written. Unchanged entries are copied raw, stored ones stay stored.

Grass reach

Grass meshes carry no switches; the game fades grass by distance per grass type, in initial.pak:

> <GrassBrand FadeDistances="7, 17, 30"> density steps, then the end of the grass

Most grass is gone at 30 m. The card multiplies the distances by its factor, with one limit: the game draws grass only to about 64 m from the camera, and a step set beyond that no longer fades, so the grass at the edge appears whole at once (found by Milliath). Step n of k therefore ends at most at

> $d'_n = \max(d_n, \min(f \cdot d_n, 60 - 5\,(k - 1 - n)))$

which gives 21, 51, 60 m at 3 and 14, 34, 60 m at 2 for the common "7, 17, 30"; a step the game itself sets farther (some brands end at 80 to 150 m) stays. Cost: more grass blades drawn, which on dense meadows is the card's real cost.

Map days

Card: Map days. Which region's time-of-day states and sky each map cycles through.

Where a map names its days

Each level's pak (level_us_06_01.pak) holds [scene]\data_for_ssl.bin, stored, which begins with its names, each a 16-bit length, the text and a zero: the level's name, its description, then the states of the night, the dawn, the three days (each a ; list of a state and its weather variants), the dusk, and the region's sky file. Four regions ship no states of their own and name another region's: Maine names Yukon's and Alaska's, British Columbia Michigan's, Austria Belozersk's and North Carolina's, Washington North Carolina's. The game takes the names as they stand: the file's own check value is not compared (tested in the game), and any state initial.pak knows can be named. A name it does not know stops the load.

The edit

Every region code in the head is replaced by another of the same length (us_04 → us_01), in the player's own file, in place. The zip's CRC of the entry follows in both of its places, the local header and the central directory. Nothing is kept for the way back: Restore writes the game's codes, which the tool's table holds, and the CRC comes out as it was. A level file another mod changed is left as it is. Apply and Restore check that the level files can be written only when they may write them (Map days chosen or in), so a read-only level file stops no run that leaves the maps alone.

The choices

Each level's snow and ice textures were counted from its load list (Yellowrock National Forest has two to four times the snow and ice of The Lowland, which is at Michigan's level), and each region's season taken from the wiki and checked against the game (Scandinavia, filed under spring, is late winter there, with snow). The map by map value then picks per level: The Lowland Michigan's days, Yellowrock Yukon's spring with its blizzards turned to flurries, Duncan Bay Wisconsin's summer with its heavy rain, North Peak North Carolina's, Unterfeld Tennessee's, Washington Michigan's autumn and its Mountains Zurdania's. Every target region has all ten state names each level uses.

Sets of a map's own

Maine's two maps get sets of their own, us_91 and us_92, numbers no region of the game uses. A set is made from a source region's states and sky as they stand in the build: the files under the new names, the state names inside the sky file (DayTimeOverride DayTime="day__1_us_04") renamed with them, and the weather changed by a table (Yellowrock: heavy_snow becomes light_snow at 0.4 of its intensity). The engine adds the 20 files to initial.pak, stored, at the end, with their directory records where their names belong; then it names every file in the pak's load list, without which the game does not know them (next chapter). A level file names a set only while initial.pak holds it: the tool puts the set in before writing the level files, and takes it out after none names it. If a game update replaces initial.pak while a level still names a set, the status reads "part" and the next Apply mends it. Restore keeps initial.pak as it is while a level file still names a set: with one of Maine's two files changed by another mod the level files are not put back, and the game's own initial.pak, which holds no such set, would stop the other map from loading; Restore says so instead.

The load list: how a pak tells the game what it holds

The engine part behind Map days' own sets, and behind any file added to a pak.

The first entry of every pak, pak.load_list, lists everything the pak loads and in what order: a start, stages ("TEMPLATES load", "CLASSES load", "TEXTURE load", "MESH load"), the assets between them, an end. A class file (a truck, a daytime state, a sky) that is in the pak but not in this list is unknown to the game: a level that names it crashes. That was tested in the game: twelve states added to initial.pak without list entries crashed the level; the same states with entries loaded.

The layout follows the reading in SnowRunnerTools by chase-000:

Part	Bytes
head	i32 1, u8 1, i32 count, i32 3, u8 1
types	count bytes: 5 the start, 1 a stage, 2 an asset, 6 the end; then u8 1
waits	per entry: i32 n, u8 1, n × i32, the entries it waits for (counted from 0, ascending); then u8 1
strings	per entry: i32 s, i32 2, s + 2 bytes of 01, then s strings (u32 length and text)

An asset has three strings (its name, its loader, its pak: "<media>\classes\daytimes\x.xml", "cls_loader", "initial.pak") or four (a texture in a level pak: a JSON text naming its base). The rules the game's own lists keep: an asset waits for one entry (the stage before its run, or, for a texture's faces, its header); a stage after assets waits for exactly those nothing else waits for; a stage after a stage waits for that stage; the class files stand together in the byte order of their names. The tool reads all 73 of the game's lists and writes them back byte for byte. Adding a class file means: the new entry where its name belongs among the class files, waiting for the stage before them; every number naming a later entry raised by one; the stage after the class files waiting for the new one too.

Next Gen logo and the start without the intro

Card: Next Gen logo (row Start-up).

The logos

NEXT GEN appears under the game's logo on five textures in gfx.pak. Three are pictures of their own (the title screen, the first loading screen at 3840 × 2160, the main menu) and are replaced whole. Two are sheets of many sprites (the pause menu, 500 × 504, and the map screen, 512 × 508), where only the logo sprite should change, so that an interface mod's other sprites stay. The sheets are BC7: every 4 × 4 block of texels is 16 bytes of its own, independent of its neighbours. So the tool copies exactly the blocks inside the logo sprite's rectangle (multiples of 4 texels) from Next Gen's sheet into the sheet the pak holds, and every other block stays bit for bit. A sheet of another size or format is left as it is.

The start without the intro

The game's intro is not a video: one Flash (Scaleform) movie in gfx.pak's [gfx]\gfxbundle.gfxbundle runs it, at 35 frames a second. Frames 2 to 259 show the epilepsy and legal warning (7.4 s); a script on frame 259 stops the movie and tells the loading screen it is ready (OnSkippableReadyToStart); the loading screen then calls StartSkippables, which plays the publisher logos (about 11 s, skippable by a key); at their end OnAllAnimFinished hands over (OnLogoEnd) and the bootup loading screen shows: the Next Gen splash. The row edits the movie's ActionScript bytecode, in the movie and its Japanese twin:

Where	Game's bytes	Next Gen's
the constructor's addFrameScript	pushshort 258 (25 82 02)	pushbyte 9, nop (24 09 02): the "ready" script runs at frame 10
StartSkippables (38 bytes)	focus, input listener, gotoAndPlay("skippable")	nops, then this.OnAllAnimFinished(): the logos end at once

Each edit keeps its size, so the bundle's header (which lists the movies' names and lengths, no offsets or checksums) stays valid; only the pak's zip CRC of the entry changes. Each edit is found by the whole method's bytes as the game ships them; a game version whose movie differs keeps its intro. Measured: the title screen came 41 s after start with the intro and 24 to 25 s without (warm disk cache). The loading behind the intro still has to finish; the saving is the part of the intro that outlasted it.

The engine: building, checking, undoing

What happens when you click Apply or Restore. The window hands a selection (the ticked cards and their rows) to the engine, a set of Node.js tools that ships in the download's engine folder.

The paks are zips

Each pak is a plain zip: entries back to back, stored or deflate, no zip64 in the paks Next Gen touches, padded with zeros to a multiple of 4096 bytes behind the end record. Some of them are large (shared.pak 2 GB, boot.pak 1.1 GB, gfx.pak 0.7 GB), so a writer copies unchanged entries raw, in large chunks, and writes only what changed. A changed entry gets a new local header (its CRC-32 and sizes), the central directory gets every entry's new offset, and added entries go after the last one.

Always from the original

Before its first change to a pak the engine keeps a copy of it in %LOCALAPPDATA%\SnowRunnerNextGen (one folder per game install), checked by hash after the copy. Every build starts from that copy, never from the file in the game: a build is a function of the original and the selection, so applying twice gives the same file, and taking a card out is the same as never having had it. Several cards can share one pak: initial.pak, for example, carries grass reach, fill light, brighter stars, weather, debris and Map days' own sets, in one build.

Write, read back, then replace

A new pak is written next to the old one (name.tmp), read back and checked: the same entries in the same order, every changed entry inflated and compared with what was planned, every other record unchanged, and every 50th untouched entry read and its CRC checked. Only then does it replace the old file, by a rename after a flush to disk, so a crash or a power cut leaves either the old file or the new one, never half of one. The engine refuses to write while SnowRunner.exe runs; it asks Windows for the game's process in UTF-8, so a game folder with letters outside the console's code page (D:\Игры\SnowRunner) is recognised too.

Knowing its own builds

For each pak the engine notes (in a small JSON next to the kept original) the SHA-256 of the build's central directory, its size and what went into it. Hashing the directory is enough to recognise a file: every entry's CRC-32, size and offset is in it. So the status of a pak is one of four:

State	Meaning
ours	the file is the engine's last build (its directory hash matches the note); the note says which cards are in
vanilla	the file is the kept original
changed	neither: another mod or a game update wrote it since
missing	no such file

The note is written in two steps around the replace (a pending entry first, the final one after), so a build interrupted between the two is still recognised. A pak no tool can read (a zip64 file, bytes behind its end record, more than 65535 entries: another tool's layout) reads as changed, and the log says why: status, Apply and Restore go on with every other file instead of all stopping. Only such a layout counts: any other failure of a tool (a lock, a full disk) stops as before, since a yes to the question that follows would keep the engine's own shared.pak or shader.pak as the original.

The engine's own state for a game (the kept originals, the notes) is named after the game's folder as it really is, so the same game reached through a junction, a subst drive or another drive letter finds it. Before a first copy of an original it checks that the drive has room for it, with 256 MB to spare, and a copy that fails on the way leaves nothing half written behind. Only one Apply or Restore runs on a game at a time, held by a lock file with the process's number; a lock whose number now belongs to another program, or older than six hours, is taken over.

The skies are a download of their own (the sky pack). Every change to boot.pak builds it again from the original with the parts already in, so a new version unzipped into a new folder without the sky pack still keeps the skies: they come from the boot.pak in the game, the engine's own build. Restore takes the skies out first. The pack carries a list of its files and their sizes (pack.json): a pack an unzip left short, the list itself included, is told apart, and Apply leaves the skies out and says to unzip it again instead of writing the textures it has. With the skies in and the pack not there, a yes to taking a changed boot.pak as the original is refused until the pack is back: the last build's skies are told from the other mod's changes by the pack's own files, and without them the whole last build would stay in the new original, out of Restore's reach. A texture or LUT a game update renamed or took away is left out with a warning instead of stopping every Apply at the same step, and a shader module the game's shaders can no longer take is remembered as left out, so the next Apply does not build shader.pak again for it.

Other mods and game updates: adopt

When a pak is "changed", Apply asks. "Take it as the original" adopts the file: the engine walks its entries, and every entry that still holds exactly what the engine's last build wrote (same CRC, same size, same bytes) goes back to the old original's bytes. What the other mod changed stays. The result becomes the new original (the old one is kept under a dated name), and the build runs over it. Restore then puts that modded state back, the other mod's changes included. "Leave it out" keeps the file and its cards as they are.

A game update replaces the paks, so after one the game shows its own look until Apply has run again, and Apply asks the same question. The game update of 20 October 2026 (exe build 6ab52f5d, a new region) was tried that way on copies of the new files: every card applied with no warning. The engine also knows the game's own shader.pak by its hash, for July's version and that update's, so a first install takes it as the original without asking. One part of SnowRunner Shadows reads the game's code itself, the texture base rule. The game keeps a texture's smallest mips in a block whose start it works out from one value, 512, and upscaled textures with sides like 192 only load when that value is 256. On the build it was made for (6a607c05) the DLL sets it at load, at a known address. On any other build it looks for the value when the game makes its Direct3D device, before any texture exists: through the one instruction sequence that reads it (mov ecx, [value]; mov eax, ecx; cdq; sub eax, edx; sar eax, 1), in the game's code, pointing at an int of 512. If none is found, or the readers point at two different values, it leaves the game's rule alone and says so in its log.

The note in the game folder

Next to the paks the engine leaves SnowRunnerNextGen.json, naming the files that hold its changes with their directory hashes. If the kept originals are ever lost (a new Windows install with the game on another drive), the engine can still see that those files are not the game's own; it then changes nothing and asks for the store's file check first, instead of keeping a modded file as the "original". Restore takes the note away.

The shaders

The first Apply reads the game's own shader cache and prepares the shader cards from it (a minute or two): every patch tool runs over the stock shaders, and what it made is noted with its hashes. Later Applies

combine the prepared modules for the ticked cards and write shader.pak in one pass. The DLL, hid.dll, is copied next to SnowRunner.exe with its settings file; another mod's hid.dll is kept as hid_chain.dll and still loaded.

Tests

Every change to Next Gen passes a test chain before it ships: the window's tests (the cards, the rows, the selection it sends), the engine's tests on stand-in game folders built from real originals (every card on, off, changed by another mod, adopted, restored byte for byte), the package, and the packaged engine run again. The last full chain on this version passed 454 checks.

Known issues and fixes

What writing this guide turned up, what was fixed, and the limits that are part of a technique's design rather than bugs, so they are not reported as such.

Fixed

What was wrong	What changed	Commit
The Map days help said three regions have no days of their own; Washington, the fourth, was missing.	The help names four regions and Washington's row.	1c8ad5e, 2026-10-17
The README described Map days without Washington and without the map-by-map values, the logo card without its Start-up row, and the files it changes without Maine's own sets, the start without the intro and Washington's level files.	The README lists them.	1c8ad5e, 2026-10-17
The window's list of keys in the game (How it works, Comparing in the game) named F4, F5, F6, F8, F10 and F11 but not F9, which switches the reflections on wet ground.	The list names F9.	d01a6b5, 2026-10-18
Tool notes out of date: the weather tool said it has five parts (it has nine); lod_patch.js said the weather default is all seven parts (it is every part but land, eight), and its usage lacked debris-install, days-set and the debris= and days= options of initial-build.	The notes and the printed usage say what the tools do.	d174ccd, 2026-10-18
A player saw an old picture, the menu's, in the shadows after changing a video setting in the game, until F8 or a restart. The scene copy that the bounce light and the reflections read stayed bound when the copies stopped, and kept its last picture.	The copy is bound only within 250 ms of the last one; otherwise every reader gets nothing and keeps the game's own result. Not yet seen in the game.	2026-10-21

Each fix passed the full test chain (454 checks) before it was committed. None of them changes anything in the game.

Limits by design

Card	Limit
Object reflections, Water reflections, Wet ground reflections, Contact shadows, GTAO, Bounce light	Screen space: they know only what the camera sees. Things off screen or hidden behind something are not reflected, cast no contact shadow and no occlusion; reflections fade at the screen's edges.
Object reflections, Wet ground reflections, Bounce light	They read the previous frame, so they lag one frame and have nothing for pixels just uncovered.
Bounce light (off by default)	In places, bright snow next to dark wood for example, it comes out too strong.
Volumetric fog, Sun shafts (off by default)	The fog does not know what stands in front of each step, so with a low sun the shafts also show over the truck and other things close to the camera.
Water reflections, Sun glitter in shade (off by default)	Where it was tried the game's own shadow test said the water was lit, so nothing changed.
Sun glow through waves	8 of the 20 river shaders do not read the sun's shadow map and get no glow.
Shadow edges, seam dither	The band where cascades blend relies on the game's TAA; detail smaller than a shadow texel cannot be rebuilt.
Map days, map by map for Maine	Maine's own sets live in initial.pak. If a game update replaces it while Maine's level files name the sets, Maine does not load until Apply has run again.
Next Gen logo, Start-up	The game still loads behind the splash; the time saved is the part of the intro that outlasted the loading (about 16 s in the test).

Part 5. What it changes on your PC

All under the game's **preload\paks\client** folder, and only those that a ticked card needs:

File	What Next Gen puts in it
shader.pak	the shader modules: shadows, occlusion, light, reflections, water, fog, smoke, the tonemap
shared.pak	Scenery detail
initial.pak	Grass reach, the fill light, Brighter stars, Weather, Debris and splashes, and Maine's own days (Map days, map by map)
boot.pak	Photo grade, Sharper smoke and particles, Night sky, Reflection cubes, Winter skies
gfx.pak	Next Gen logo, and the start without the intro
level files	with Map days, a few names in the level files of Maine, British Columbia, Austria and Washington, changed in place

In the folder with SnowRunner.exe (Sources\Bin on Steam): **hid.dll**, which is SnowRunner Shadows, and its settings, **SnowRunnerShadows.ini**. Another mod's hid.dll stays loaded as hid_chain.dll.

The copies of the original files live in **%LOCALAPPDATA%\SnowRunnerNextGen**. Every new file is written next to the old one, read back and checked before it takes the old one's place. Next to the paks the program leaves a small note, **SnowRunnerNextGen.json**, naming the files that hold its changes; Restore originals takes it away.

Part 6. Licence and credits

Next Gen is free software under the GNU General Public License, version 3 (the file LICENSE in the download). The methods and code it builds on are named in **THIRD_PARTY_NOTICES.md**, with their authors and licences, and each card's page above credits the work it uses. The game's files remain Saber Interactive's: Next Gen changes them on your PC and ships none of them, apart from the few textures the notices name.